

Dr. A.P.J. Abdul Kalam Technical University, Lucknow



B. Tech. Program

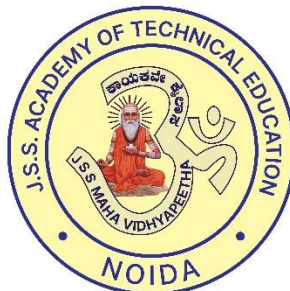
Project report on

***LoRa based Dust and
weather monitoring system***

*Submitted
by*

<i>Name</i>	<i>Roll Number</i>
<i>Gokaran Dev Gautam</i>	<i>2000910310070</i>
<i>Hrithik Yadav</i>	<i>2000910310079</i>
<i>Vaishnavi Agrawal</i>	<i>2000910310181</i>

Under the Guidance of
Dr. Sadhana Kumari
Assistant Professor



June, 2024

***DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
JSS ACADEMY OF TECHNICAL EDUCATION
C-20/1 SECTOR-62, NOIDA, UTTAR PRADESH.***

Dr. A.P.J. Abdul Kalam Technical University, Lucknow



B. Tech. Program

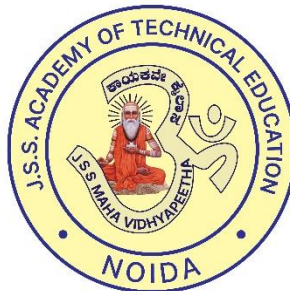
Project report on

***LoRa based Dust and
weather monitoring system***

*Submitted
by*

<i>Name</i>	<i>Roll Number</i>
<i>Gokaran Dev Gautam</i>	<i>2000910310070</i>
<i>Hrithik Yadav</i>	<i>2000910310079</i>
<i>Vaishnavi Agrawal</i>	<i>2000910310181</i>

Under the Guidance of
Dr. Sadhana Kumari
Assistant Professor



June, 2024

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
JSS ACADEMY OF TECHNICAL EDUCATION
C-20/1 SECTOR-62, NOIDA, UTTAR PRADESH.**

DECLARATION

We hereby declare that this submission is our own work, and that to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher, except where due acknowledgment has been made in the text.

1. Hrithik Yadav
Roll No: 2000910310079

2. Vaishnavi Agrawal
Roll No: 2000910310181

3. Gokaran Dev Gautam
Roll No: 2000910310070

CERTIFICATE

This is to certify that Project Report entitled “LoRa based Dust and Weather Monitoring System” which is submitted by GOKARAN, HRITHIK & VAISHNAVI AGRAWAL for partial fulfillment of the requirement for the award of B. Tech degree in Electronics and Communication Engineering of Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of the candidates own work carried out by them under our supervision. The matter embodied in this thesis is original and has not been submitted for the award of any other degree.

Dr. Sadhana Kumari
Assistant Professor

LoRa based Dust and weather monitoring

ORIGINALITY REPORT

12%

SIMILARITY INDEX

11%

INTERNET SOURCES

7%

PUBLICATIONS

5%

STUDENT PAPERS

PRIMARY SOURCES

1	www.coursehero.com Internet Source	2%
2	forum.arduino.cc Internet Source	2%
3	circuitdigest.com Internet Source	1%
4	how2electronics.com Internet Source	1%
5	Submitted to Hogeschool Rotterdam Student Paper	1%
6	www.researchgate.net Internet Source	<1%
7	create.arduino.cc Internet Source	<1%
8	www.theengineeringprojects.com Internet Source	<1%
9	forbot.pl Internet Source	<1%

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the B. Tech Project undertaken during B. Tech. Final Year. We owe special debt of gratitude to Dr. Sadhana Kumari, Assistant Professor, Department of Electronics and Communication Engineering, J.S.S. Academy of Technical Education, Noida for his constant support and guidance throughout the course of our work. His sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only his cognizant efforts that our endeavors have seen light of the day.

We also take the opportunity to acknowledge the contribution of Dr Arun Kumar G Head of Department of Electronics and Communication Engineering, J.S.S. Academy of Technical Education, Noida for his full support and assistance during the development of the project.

Signature:

Name: Gokaran Dev Gautam

Roll No: 2000910310070

Signature:

Name: Vaishnavi Agrawal

Roll No: 2000910310181

Signature:

Name: Hrithik Yadav

Roll No. 2000910310079

ABSTRACT

In the present age of Swachh and Smart Bharat Abhiyan, our aim is to provide our college with necessary tools to monitor factors like PM2.5, PM10, CO, CO2 and other pollutants which affect the vitality of people by causing air pollution. Using our project, we will be able to interface various sensors and use the data to monitor the aforementioned parameters along with the weather conditions, specifically in a localized area. Using the data from temperature, humidity, and pressure sensors, we will also be able to calculate and display dew point, wind direction, wind speed, rainfall measurement.

At the heart of this project is Lora WAN protocol, using which we will be able to transmit the collected the sensors data over a range of over 1 kilometer.

TABLE OF CONTENTS

	Page
DECLARATION	ii
CERTIFICATE	iii
ACKNOWLEDGEMENTS	iv
ABSTRACT.....	v
LIST OF FIGURES	viii
LIST OF TABLES	viii
LIST OF ABBREVIATIONS.....	x
CHAPTER 1 LITERATURE SURVEY	1
1.1 Literature survey	1
1.2 Literature overview	5
CHAPTER 2 INTRODUCTION, AIM & OBJECTIVE.....	13
2.1. Introduction of project	13
2.2. Problem Statement.....	18
2.3 Aim and Objective	18
CHAPTER 3 PROPOSED ALGORITHM	20
3.1 Block Diagram	20
3.2 Embedded System Implementation.....	20
3.3 Embedded System Software.....	22
3.4 Transmission side model	30
3.5 Receiver side model	37
3.6 Overall integration of project	38
CHAPTER 4 RESULT AND WORK DONE	53
4.1 Result & implementation.....	53
CHAPTER 5 CONCLUSION AND FUTURE SCOPE	54
REFERENCES.....	56

LIST OF FIGURES

	Page
FIGURE 1 Three tier architecture of IOT	6
FIGURE 2 Proposed system architecture.....	7
FIGURE 3 AI Thinker RA02	16
FIGURE 4 ESP32.....	17
FIGURE 5 0.96 OLED display	18
FIGURE 6 Block diagram of system	20
FIGURE 7 Flow of flash code to processor	22
FIGURE 8 Arduino-IDE download page	24
FIGURE 9 Arduino IDE startup.....	25
FIGURE 10 Action panel.....	26
FIGURE 11 Preferences window	27
FIGURE 12 Boards manager	27
FIGURE 13 Screen to add additional boards	28
FIGURE 14 Boards manager	28
FIGURE 15 MQ unified library	29
FIGURE 16 Defining properties of MQ sensor	30
FIGURE 17 Output for MQ sensor	30
FIGURE 18 Properties for MQ4 sensor	31
FIGURE 19 Output for MQ4 sensor	31
FIGURE 20 Serial communication.....	32
FIGURE 21 Serial communication Temperature and humidity	32
FIGURE 22 Temperature and humidity data using serial communication	33
FIGURE 23 Script for logging data into google sheet	34
FIGURE 24 Random values on spreadsheet	34

FIGURE 25 Temperature, humidity and AQI on spreadsheet	35
FIGURE 26 DSM501A dust sensor	35
FIGURE 27 Non converted values of dust sensor.....	36
FIGURE 28 LoRa library by Sandeep Mistry	37
FIGURE 29 Sending data using LoRa	39
FIGURE 30 OLED testing	42

LIST OF ABBREVIATIONS

IoT		Internet of Things
M2M		Machine to Machine
AWS		AmazonWeb Services
ITU		International Telecommunication Union
LoRaWAN		Long Range Wide Area Network
ESP		Espressive System
PM		Particulate Matter
OLED		Organic Light Emitting Diode
CSS		Chirp Spread Spectrum

CHAPTER 1

LITERATURE SURVEY

1.1 LITERATURE REVIEW

Harsh Gupta, et al. [1] A remote server or an Android application can be used to visualize the data transferred in real-time while air quality is being monitored. This requires live data communication between the devices via the internet. Since there is only one installation expense, it minimizes the mobilization of system hardware at several locations, which is cost-effective. Environmentalists and climate scientists are very concerned about air pollution, climate change, and its effects in the modern world. Children, asthmatics, expectant mothers, the elderly, and other vulnerable members of society are all at serious risk from poor air quality. According to WHO statistics, air pollution causes millions of cases of early deaths globally each year. Thus, the right detecting and tracking system is crucial for achieving effective **IoT** accomplishment for an application. Digital control over the physical equipment is possible, and Machine to Machine (M2M) communication is a means of communication between the devices. One option could be an Internet of Things (IoT)-based monitoring system.

Priya Kurade, et al. [2] Air condition monitoring and reporting system is used how people today want to get the most recent information on the weather wherever they are. In order to provide individuals with easy and quick access to the most recent information on weather conditions, we are building a weather monitoring system. Many weather monitoring systems have been developed recently to address various environmental factors. The Condition Monitoring and Reporting Framework for Wireless Sensor Network-based AWS is one of the current system models. A framework for monitoring AWS conditions is included in this suggested system to help with preventive maintenance. It has been determined from earlier study that once daily manual observations and once-hourly automatic observations were conducted in the past.

However, with today's advanced technologies like Arduino and IOT, taking observations at any time and from any location is quite easy. The current system model is based on the Arduino system, which allows measurable data to be collected, analysed, and presented on a web server. The data is also accessible through internet connectivity on mobile phones, web servers, and other web browser devices, which improves the system's intelligence and ease of use.

Evariste Twahirwa, et al. [3] The authors created an Internet of Things (IoT) cloud with many server types, allowing sensed data to be presented via mobile or online apps. By displaying both historical and real-time data, this study highlights how IoT may be used to build pollution-sensitive work environments and assist homeowners in detecting exceptionally high indoor air pollution levels. Installing air quality monitoring systems with efficient operating procedures in residential buildings is critical for creating sustainable, pollution-free living environments and improving lives. According to WHO estimates, air pollution-related illnesses caused more than 8 million deaths in 2016. Furthermore, 91% of the world population lives in places where pollution levels exceed WHO standards.

Ravi Sai Manasa, et al. [4] LoRa Module: LoRa, which full form is "Long Range," is a physically based, exclusive radio communication method. The chirp spread spectrum (CSS) approach serves as the foundation for spread spectrum modulation techniques. It was created by cycle (patent 9647718-B2), a Grenoble, France-based business that Semtech eventually purchased. The system structure and protocol for interaction are defined by LoRaWAN. A recognized ITU-T Y.4480 standard, LoRaWAN is administered by the International Telecommunication Union (ITU). SemTech is a founding member of the open, independent LoRa Alliance, which oversees the ongoing development of the LoRaWAN protocol. In regional, national, or international networks, LoRa and LoRaWAN together describe wirelessly connecting battery-operated devices to the internet. These sensors use individual sensors to collect data from the environment, such as the climate, and send each one of the information to ESP32. Whereas the recipient module will receive this data from the ESP32 via the LoRa portion. The battery that we fixed with the ESP provides electricity to the entire framework. There are only two components: ESP32 and Lora. The ESP32 WiFi Module ships the data off the web, while Gateway-Node LoRa receives the provided information from LoRa. The client can verify or notice the information here thanks to the Web software. For remote

communication, a LoRa-based weather station needs a sender and receiver circuit. We are able to maintain the Weather Station framework atop your house or in any remote location that is only a few kilometres away. In essence, this weather station may filter parameters related to the environment, such as temperature, humidity, pressure, CO, CO₂, PM_{2.5}, and PM₁₀ particulates in the atmosphere.

G. Jayashree, et al. [5] This research presents an autonomous real-time environmental condition monitoring system for air quality. IoT technology and embedded systems provide the foundation of this suggested solution. This system also has electronic components, such as an Arduino Uno board for system performance and sensors to sense the surroundings. This system incorporates a LoRa module for long-distance communication. While Bluetooth, Zigbee, and Wi-Fi are still viable options, this system also incorporates LoRa for long-distance communication. The major goal of this project is to monitor smart city environmental parameters including temperature, humidity, rain, and pressure. It also aims to detect light and air pollutants. The system is based on LoRa technology. These environmental parameter values are kept on a cloud server that communicates with the LoRa Gateway. The cloud server provides data monitoring capabilities to the user, with values shown in the form of tables or graphs.

S M Abiduzzaman, et al. [6] The goal of this project is to provide real-time location and information about air quality by creating a mobile app and a handheld device that can generate and transmit real-time geographic coordinates and the Air Pollutant Index (API). This paper has reviewed a number of air contaminants as well as issues with the current air quality monitoring system. The technology determines the value of the Air Pollutant Index (API) and uploads the geographical data to the cloud library. Everyone with the built smartphone app can access real-time information about the location and air quality. The application also retrieves the data.

Virendra Barot, et al. [7] We provide a thorough analysis of potential enabling technologies for Internet of Things-based AQMS framework in this article. In particular, we begin with a summary of the main classes of inexpensive air pollution sensors, common sources of error, and calibration techniques. Next, in order to facilitate IoT-based design for AQMS, we give an analysis and comparison research of application layer and infrastructure protocols.

Additionally, we reviewed the current systems and categorized them according to the deployment approach used.

Indranil Sarkar, et al. [8] The creation of a prototype weather device for monitoring temperature, relative humidity, atmospheric pressure, wind direction, speed, and rainfall is the primary goal of this publication. The purpose of this kind of weather device is to measure weather data autonomously. The measured data is wirelessly sent to a distant station for recording along with data display on various smart devices. Wi-Fi connections have been used in the planning of this wireless connectivity, creating a mesh network for dependable data transfer. Additionally, the residents are assisted in taking the appropriate safety measures by our reliable outputs.

Toju Babalola, et al. [9] The main objective is to design a smart device at a low cost that can store data that is collected from the measurement of different atmospheric physical factors, such temperature, humidity, and rainfall. Measured data is stored using an ESP-32, which is a sophisticated and effective Internet connectivity option. The direct environmental sensing, data measurement and storage, and cloud-based result access are the three main components of the intelligent weather station system. A microcontroller that functions as the system's server and central component, along with portable internet communication protocol, make up the foundation of the system. The outcomes were uploaded to the cloud and are shown there in a graphical and tabular style. These outcomes can be downloaded into an Excel file from the cloud.

Eko Murdyantoro, et al. [10] The creation of an atmospheric monitoring model utilizing LoRa wireless technology is the aim of this research. LoRa is a wireless communication technology that underpins the Internet of Things (IoT). Other commonly used wireless communication modules, such as GSM, Wi-Fi, and Bluetooth (BLE), are replaced by this technology. The LoRa network is used to increase the range of wireless cells, which may travel up to 5 kilometres at low energy usage. A single LoRa network cell can link end devices to hundreds of nodes. Measured meteorological parameters include temperature, humidity, air pressure, rain detection, and wind speed.

1.2 OVERVIEW OF LITERATURE SURVEY

1.2.1 IoT and LoRa Technologies

As technology and sensors have evolved, the Internet of Things (IoT) has arisen as a low-cost alternative to expensive, stationary air quality monitoring stations for establishing air quality management systems (AQMS). Low Power Wide Area Network (LPWAN) applications make use of the LoRa protocol for IoT, which allows for large-scale wireless sensor networks (WSNs). LPWANs are perfect for smart city applications because of their low cost, long range, and energy economy. These developments provide a new communication paradigm for discrete IoT-applications.

LoRa, a more recent LPWAN technology, employs a broad band broadcast approach that makes it resistant to fading, Doppler effects, channel noise, and long-term frequency fluctuations. LPWAN is a fast expanding part of the networking business.

indoor and outdoor settings has drawn more attention in recent years. An air quality monitoring system with a three-tiered low power wide area network (LPWA) architecture is created and put into use.

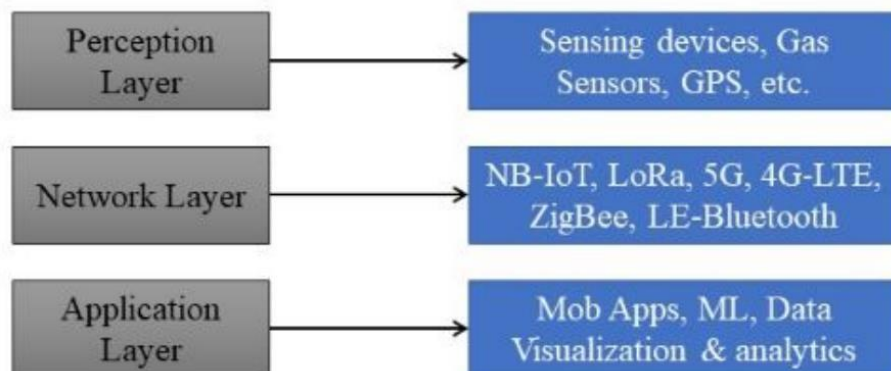


Fig. 1. The three-level architecture of Internet of Things

1.2.2 Air Quality Monitoring System

Sensors, a gateway, a cloud, and end users make up the suggested and implemented air pollution tracking system. For information on the system architecture, see Figure 2. The sensors that comply with LoRa WAN are in charge of detecting the air pollutants present in the atmosphere. The sensing devices that have been installed are capable of withstanding

extended periods of high humidity and high temperatures. They function within a spectrum of temperatures of -10 to $+60^{\circ}\text{C}$ and a non-condensed humidity range of 0 to $85\% \text{ RH}$. The gateway that connects the sensors to the cloud portion of the network and supports the LoRa WAN protocol is then used to transmit the measured parameter readings to the cloud. End users have access to the data and analytical information in the cloud and are able to query it. When the amount of contaminants in the air surpasses the WHO-specified thresholds, an algorithm is put into place to transmit the notifications.

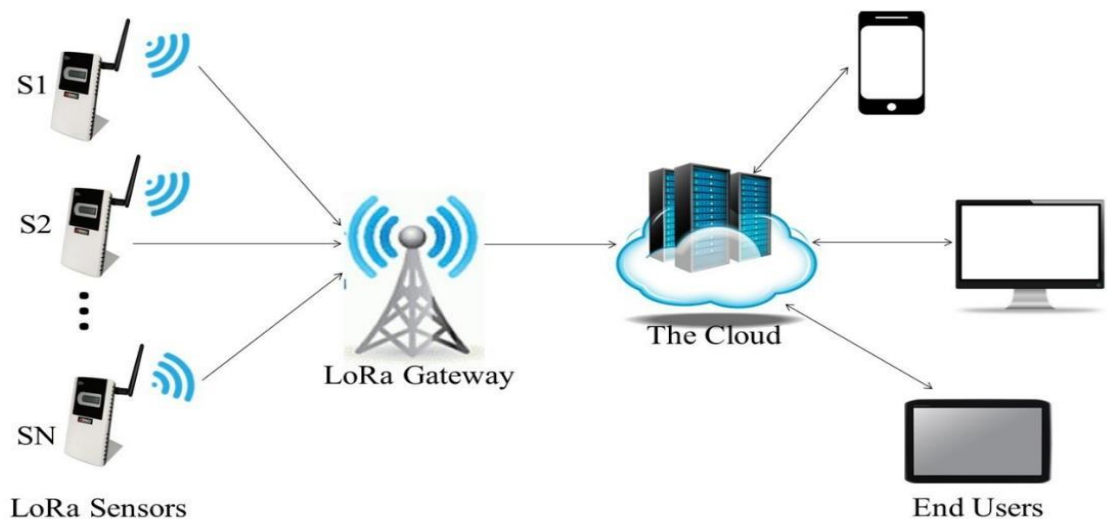


Fig. 2. Proposed system architecture: The deployed sensors are connected to the LoRa gateway through MQTT.

Air pollution, caused by microscopic, invisible particulates, is a natural part of the Earth's atmosphere. It poses a significant health risk to victims. To reduce its negative effects, real-time monitoring and reporting of air quality measures can be achieved through the use of low-cost sensors and advancements like IoT and LoRa.

1.2.3 Existing methods for air quality monitoring

1) Ground-Based Monitoring Stations: These are stationary facilities that are outfitted with a range of sensors to detect volatile organic compounds (VOCs), particulate matter (PM), carbon monoxide (CO), nitrogen dioxide (NO₂), sulphur dioxide (SO₂), and ozone (O₃). These stations, which are frequently run by government organizations, offer data from ongoing monitoring.

- 2) Mobile Monitoring: To assess the amounts of pollutants at various sites, vehicles fitted with air quality monitoring equipment are used for mobile monitoring. When evaluating the quality of the air in locations without permanent monitoring stations, this approach is especially helpful.
- 3) Satellite Monitoring: Sensing-equipped satellites can deliver useful data on air quality across wide geographic regions. They can offer spatial and temporal data on air quality as well as measure different pollutants
- 4) Low-Cost Sensors: Thanks to developments in sensor technology, a multitude of inexpensive the condition of the air sensors suitable for dispersed monitoring can now be used. These sensors can give useful information for localized air quality assessments, even if they might not be as accurate as more conventional monitoring techniques.
- 5) Crowdsourced Monitoring: Crowdsourced air quality monitoring applications have gained popularity as a result of the widespread use of smartphones and internet connectivity. Users of these apps can submit observations such as visibility and odor, or they can use the sensors built into smartphones to contribute data on air quality.
- 6) Passive sampling: The process of collecting pollutants through the use of passive collectors over a predetermined length of time is known as passive sampling. The amounts of pollutants are then determined by analyzing these collectors in a laboratory. The definition of passive sampling is frequently employed to calculate the average long-term amounts of specific contaminants.
- 7) Modeling: To simulate the dispersion of contaminants in the atmosphere, computer models are used in air quality modeling. In order to estimate air quality levels, these models take into account variables like emissions sources, weather patterns, and regional characteristics. Modeling can give information on the origins and movements of contaminants and serve as a supplement to data from direct monitoring.

1.2.4 Advantages and Limitations of LoRa technology in air quality monitoring system

Advantages:

- 1) Long Range: Because LoRa technology can travel great distances, air-quality surveillance sensors may be widely installed in both urban and rural locations.

- 2) **Low Power Consumption:** LoRa devices use less energy, allowing remote or battery-powered sensors to have longer battery lives and requiring less maintenance.
- 3) **Cost-Effective Deployment:** LoRa infrastructure is suited for scalable air quality monitoring networks since it is less expensive to set up and maintain than cellular or Wi-Fi networks.
- 4) **Penetration and Coverage:** LoRa signals have the ability to pass through structures and barriers, providing dependable connectivity in difficult urban settings.
- 5) **Scalability:** A significant number of sensors may be simply added to LoRa networks, allowing for extensive monitoring of air quality over a huge area.
- 6) **Data Security:** The reliability and safety of gathered air quality data are guaranteed by the LoRaWAN protocol's full encryption and verification.

Limitations:

- 1) **Restricted Bandwidth:** Because LoRa technology uses unauthorized radio bands, its bandwidth may be less than that of licensed spectrum. This could limit the volume of data that can be sent.
- 2) **Data Rate:** Compared to other wireless technologies like Wi-Fi or cellular, LoRa's low-power, extended design compromises data transmission speed, resulting in slower data rates.
- 3) **Interference:** Because LoRa uses unlicensed frequency ranges, other devices using the same range of frequencies may cause interference, which could compromise the reliability of data delivery.
- 4) **Network Congestion:** Network congestion can happen in places with a large density of LoRa deployments, such as densely populated cities, which can cause dropped packets or communication delays.

1.2.5 Comparison of LoRa with other wireless technologies

1) Wi-Fi vs. LoRa:

- **Range:** In general, LoRa has a greater range than Wi-Fi, particularly outside.
- **Power Consumption:** LoRa devices are good for battery-operated applications because their power consumption is generally lower than that of Wi-Fi devices.

- Data Rate: Wi-Fi is more appropriate for applications needing high-speed data transfer since it typically delivers higher data rates than LoRa.
- Infrastructure: LoRa uses a point-to-multipoint topology, which makes it easier to establish infrastructure in some cases, such as Internet of Things sensor networks, than Wi-Fi, which requires access points for connectivity.

2) Cellular (such as 4G/5G) vs. LoRa:

- Range: LoRa frequently has a greater range than cellular technology, particularly in isolated or rural locations with poor infrastructure.
- Power Consumption: Low frequency operation electrically powered applications benefit from LoRa devices' generally lower power consumption compared to cellular devices.
- Cost: Compared to cellular networks, LoRa networks may be less expensive to set up and run, especially for applications requiring modest data rates and a large number of devices.
- Data Rate: Compared to LoRa, cellular technologies offer higher data rates, which makes them more appropriate for applications requiring massive bandwidth transfers or video streaming.

3) Bluetooth Low Energy (BLE) vs LoRa:

- Range: Compared to BLE, LoRa typically has a longer range, which makes it better suited for applications that need communication over longer distances.
- Power Usage: LoRa and BLE are both intended to have minimal power usage, while LoRa usually uses less power when transmitting over long distances.
- Data Rate: BLE is more suited for apps that need communication in real time or frequent data updates than LoRa since it can handle larger data rates.
- Application Focus: While BLE is typically used in short-range applications like wearable devices, proximity sensing, and interior navigation, LoRa is frequently utilized in long-range IoT sensor networks, surveillance of assets, and smart agriculture.

4) LoRa vs. Zigbee:

- Range: In general, LoRa has a greater range than Zigbee, particularly in open spaces or around obstructions.

- **Power Consumption:** In general, LoRa devices use less power than Zigbee devices, especially when transmitting over long distances.
- **Network Topology:** Zigbee offers mesh networking, which enables devices to relay messages and increase network coverage, whereas LoRa uses a star-of-stars topology.
- **Data Rate:** Applications needing faster connection or more frequent data updates may find Zigbee to offer better data rates than LoRa.

1.2.6 Integration of LoRa with air quality sensors

- 1) **Choosing Air Quality Sensors:** Select the right air quality sensors according to the pollutants (e.g., particulate matter, gases like CO, NO₂, SO₂, O₃, VOCs) that you wish to monitor. Make that the sensors can connect with microcontrollers or modules that support LoRa, or that they are compatible with LoRa communication protocols.
- 2) **Hardware Integration:** Attach the development boards or microcontrollers with LoRa support to the air quality sensors. This could entail employing sensor modules that communicate via industry standards like UART, SPI, or I²C, or attaching the sensors to the analog or digital input pins of the microcontroller.
- 3) **Gathering and Handling Data:** Set up the microcontroller to periodically read data from the air quality sensors. To guarantee reliable measurements, process the sensor readings as needed (e.g., calibrate sensor data, execute averaging or filtering).
- 4) **LoRa Communication:** To wirelessly transfer sensor data to a LoRa gateway or network server, use LoRa transceivers or modules. Because of its long-range communication capabilities, LoRa is perfect for sending data on air quality across great distances, even in places with inadequate infrastructure.
- 5) **Gateway Configuration:** To collect sensor data from the nodes, place one or more LoRa gateways throughout the coverage region. These gateways serve as a link between the internet and the LoRa wireless network.
- 6) **Network Configuration:** Set up the LoRa network server to accept data from the gateways and send it to an application or backend server for additional analysis and processing. Popular protocols for LoRa networks include LoRaWAN, which offers features including encryption, adaptive data rate optimization, and authentication.
- 7) **Data Visualization and Analysis:** To visualize and analyze real-time air quality data, create a backend system or leverage already-existing platforms. This could entail

building dashboards, sending out threshold exceedance warnings, and keeping track of past data for reporting and trend analysis.

- 8) **Power Management:** If the LoRa terminals are battery-operated, make sure to optimize their energy usage by implementing power-saving measures. In order to reduce energy consumption and maintain appropriate data transmission rates, this may involve duty cycling, sleep modes, and transmission parameter optimization.

CHAPTER 2

INTRODUCTION, PROBLEM STATEMENT, OBJECTIVE

2.1 Introduction

Our mission is to equip our campus with the tools necessary to monitor air pollutants such as PM2.5, PM10, CO, CO₂, and others that are harmful to public health in the era of Swachh and Smart Bharat Abhiyan. The environment is influenced by the temperature, humidity, and wind at a location throughout a specific period of time. The environment is always changing, in general.

Occasionally, there are rainy months with snowfall and downpours, as well as dry seasons. The three primary factors that affect the weather are the sun, water, and wind. Light provides energy that has the potential to affect the water cycle in some way. The breeze carries the water fume-carrying mists to lower pressure, more advantageous areas. It pours because the mists and air make the rain fall to the soil and get heavier. It is crucial to evaluate the environment on a regular basis due to the significant impact that human activity has on it.

Using the environment data, the localized atmospheric conditions will be measured. Our project will enable us to connect a range of sensors, and we'll be able to use the data we collect to monitor not only the local weather in our campus region, but also the previously listed aspects. With the data from the temperature, humidity, and pressure sensors, we will also be able to calculate and display the dew point, wind direction, wind speed, and rainfall measurement. In addition, the data will be shown on an OLED display with a time base so that people may see the climate or weather for a certain time frame. An ESP32 will be combined with this LoRa-based weather monitoring system.

2.1.1 LoRa

LoRa is a Long Range radio system. It uses an unlicensed radio band and a technology called “Chirp Spread Spectrum” (CSS) to broadcast low-bandwidth data over remarkably large areas. Because of the low power requirements, LoRa devices are inexpensive and consume very little power, so they are ideal for outdoor applications where they can even be solar-powered.

LoRa has a very low bandwidth, typically 300 to 50,000 bits per second. If you are old enough

to remember the early dial-up modems, you're familiar with this data rate! It also is best suited for data that isn't constantly streaming, but is instead sent in chunks or bursts.

Because of these constrictions, you won't use LoRa for video or audio; Wi-Fi and Bluetooth are better choices here.

But LoRa is perfect for sensor data, where you just need to send a few bytes of information and can send it periodically. This is a common requirement; things like humidity and soil moisture don't typically change too rapidly, so you can read those sensors every 10 seconds (or even every minute) and still obtain information that is current enough to be useful.

LoRa has an incredible range for something so low-powered. I tested one of the experiments we did today and achieved a range of over two blocks using simple wire antennas and a transmitter locked in a metal box in my basement (more on that "metal box" later!). With a proper antenna setup, ranges of several miles or kilometers are possible.

The current record for LoRa range was set in the Netherlands in 2020 by Thomas Telkamp, CTO and co-founder of Lacuna Space. He used a balloon to achieve **an unbelievable range of 832 kilometers, or 512 miles!**

2.1.2 Legalities of Using LoRa

LoRa uses the Industrial, Scientific, and Medical or ISM band, which doesn't require a license for low-powered devices. You're operating within the law, providing that you don't modify your LoRa radio module to increase its output.

There are different frequencies used for the ISM band around the world, so you'll need to ensure that you are using the frequency that is legal in your area. This **Frequency Plans by Country document** by **The Things Network** will help you find the correct frequency for your area.

LoRa itself was originally developed by Cycleo; a French company founded in 2002. They focused on creating a low-cost technology that allowed low-power and long-range communications for short bursts of data.

In 2012, Cycleo was acquired by [Semtech Corporation](#). Semtech continues to develop LoRa and holds the patent for its proprietary modulation technique. To design a radio product with LoRa technology, you'd typically need to purchase chips or modules from Semtech or its partners. But you can use commercial LoRa components for your designs without infringing on patents.

LoRa technology itself is not open-source. However, the protocol built upon LoRa, known as LoRaWAN (Long Range Wide Area Network), is open-source. In short, you don't need to worry about using LoRa in your designs; the manufacturer of the LoRa radio module you use has already paid Semtech its license fee. Just make sure to use the appropriate ISM band frequency for your location.

Region	Legal Permissible Frequency Range
North America	902-928 MHz
South America	915-928 MHz
Europe	865-867 MHz
India	865-867 MHz
Asia	433 MHz

2.1.3 Some major components used

a) LoRa (Long Range): is a radio signal modulation-based wireless communication technology. Through a hardware layer connection to LoRaWAN, the gateway transmits minuscule data packets from sensors and smart devices to the operator's server. Geolocation capabilities, unlicensed frequency range, low power consumption, long-range connectivity, affordability, and deep indoor penetration for signal transmission in crowded urban settings are some of the features of LoRa networks that guarantee long-term battery life.

In our project we are using **Ai Thinker** LoRa Series **Ra-02** Spread Spectrum Wireless Module



Fig 3: AI Thinker Ra-02

The LoRa module operates between 410-525Mhz with a PA boost of +18dBm, Sniffer, and Station modes. The chip working as the core of Ra-02 and powering the device is SX1278.

SX1278 is a chip made by Semtech. It has the following features:

1. LoRaTM Modem 168dB maximum link budget
2. +20dBm - 100mW constant RF output vs. V supply
3. +14dBm high efficiency PA
4. Programmable bit rate up to 300kbps
5. High sensitivity: down to -148dBm
6. Bullet-proof front end: IIP3 = -11dBm
7. Packet engine up to 256 bytes with CRC
8. Built-in temperature sensor and low battery indicator

Another Ra-02 chip is used as a receiver in conjunction with an ESP32 and OLED display. They are programed to work like a LoRa Receiver.

b)ESP32

The ESP32 Development Board is built around the ESP WROOM32 WIFI + BLE module. It fits neatly onto a solderless breadboard and functions as a simple, small system development board powered by the ESP-WROOM-32 module. The board includes all of the ESP-WROOM-32's support circuits, such as a USB-UART bridge, reset and boot mode buttons, an LDO regulator, and a microUSB connection. Every major GPIO is accessible to developers, making it a good choice for a variety of Internet of Things (IoT) applications.

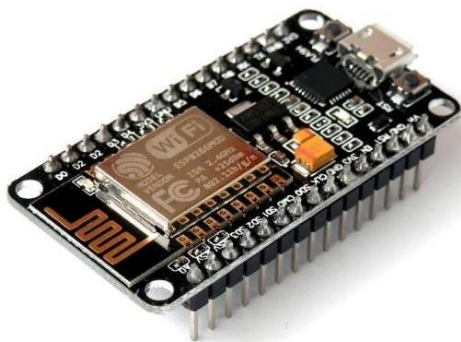


Fig. 4: ESP32

c) **SENSORS:** The instruments we use are the MQ-7 Carbon Monoxide Sensor, the MQ4 Methane Sensor, the DHT11 Humidity and Temperature Sensor, and the BM180 Air Pressure Sensor. The dust sensor, DSM510a, detects PM2.5 and PM10 particles.

d) **0.96 inch OLED:** We are using a 0.96 inch Oled screen with a resolution of 128x64 to print the receive data at the receiver side. It uses a SSH1106 chip to control the Oled and works at a voltage between 3.3-5V.

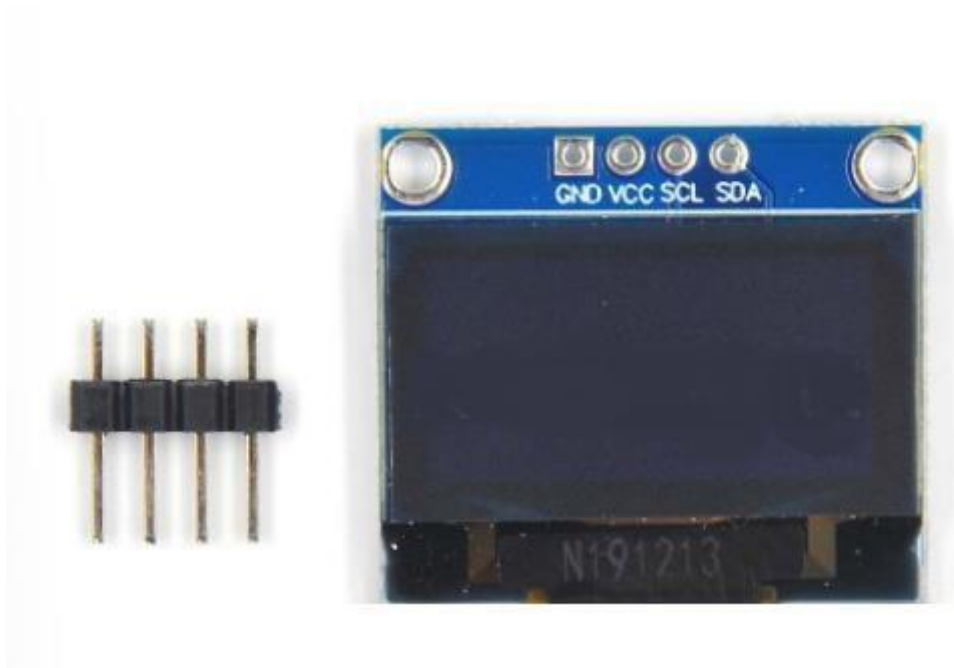


Fig 5: 0.96 Oled display

2.2 Problem Statement:

Because the environment is changing so quickly these days, internet programs' forecasts are more accurate for larger areas but less accurate for smaller ones. We believe that a weather display system that can show us atmospheric conditions (temperature, CO and CO₂ levels, AQI, humidity, etc.) for a radius of 1-2 km should be installed, just like on a college campus. We are employing the LoRaWAN protocol with a few LoRa modules to meet this special requirement; they should be able to provide us with the range at low power and good efficiency.

2.3 Objective:

Weather and AQI monitoring system with LoRa support. Measuring atmospheric parameters such temperature, atmospheric pressure, airborne pollen concentration, and CO and CO₂ levels is the goal. The three main meteorological properties—temperature, pressure, and AQI—are determined on various scales, and various components and sensors that are networked together are used to detect each quantity.

Right now, we're using two LoRa modules to create a point-to-point communication channel: Both are using the SX1278 chips. We would get the range with comparatively less power use in this way. Using this entire model, we will also be able to store the received or transmitted data in a text file or spreadsheet document.

CHAPTER 3

PROPOSED ALGORITHM

We need a system to collect data from various analog sensors. For this purpose, we will be using an Arduino UNO R3 to collect data like Temperature, Humidity, PM2.5 and PM10, atmospheric pressure, and CO gas concentration. The Data will then be transmitted using Spread Spectrum Transmission of LoRa Module. The two lora modules can either communicate using LoRaWAN or point to point. P2P is more secure and requires less energy.

3.1 Block Diagram:

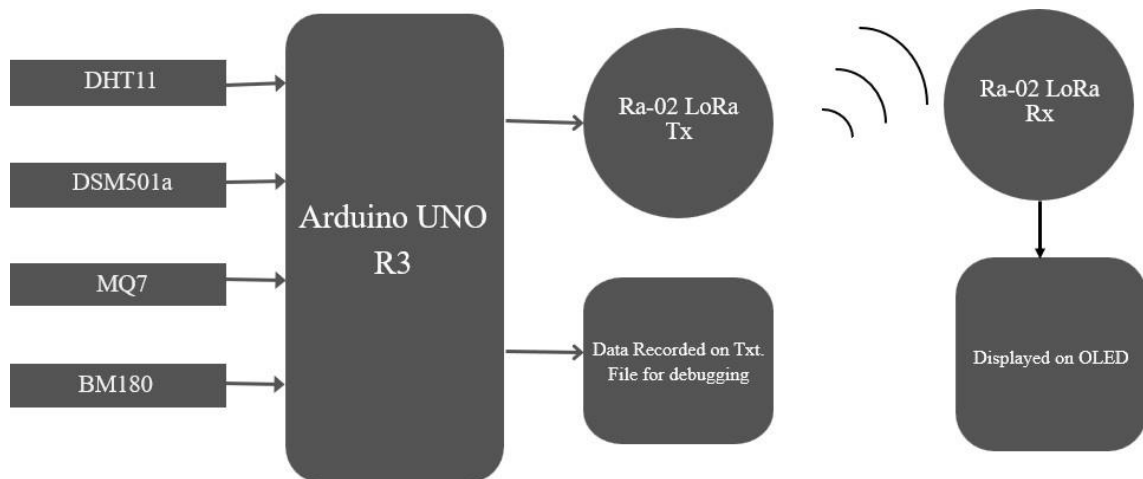


Fig 6: Block Diagram of Dust and weather Monitoring station

3.2 Embedded system implementation

Embedded systems are special computers which are designed to perform tasks like including accessing, processing, storing, and controlling data in/from different electronic devices.

These microsystems have a combination of hardware and software, commonly known as firmware. This firmware can be tweaked according to our needs and are responsible for running

things smoothly. The main objective of the systems is to work efficiently within the given constraints.

Embedded systems have an extensive list of application. From Simple smart switches to home security systems, and neighborhood traffic control systems. These electronic systems dramatically improve functionality, controllability and ease of setting up these systems

These uses demonstrate how embedded systems are widely used to improve and simplify a variety of facets of our lives. To sum up, embedded systems are essential to the seamless operation of many electronic devices, providing increased convenience and efficiency. Their applications cut across many industries and technological platforms, improving our lives at a significantly lower cost to the user.

3.2.1 Bringing hardware and software together for embedded systems

It is crucial to establish connection between software and hardware in order to extract effective functionality from the embedded system. This is achieved by flashing the source code into the microprocessor or microcontroller. The microcontroller works as a vital bridge between the input hardware and output hardware where it processes the data received on the basis of the source code flashed and then it gives us an output. While processors only run executable files, source codes for embedded systems are typically written in assembly language. There are three moves incorporated in changing the embedded software's code representation into binary image.

1. To create an object file, each source file is either compiled or run via an assembler.
2. Next, every object file that was created in the first step is synced with each other to make a unified object file that is known as the re-locatable program.
3. Thirdly, a procedure known as relocation is required to assign physical memory addresses to the relative offsets inside the relocatable application.

Below is the flowchart of the operations which are followed during the flashing of source code to a microprocessor

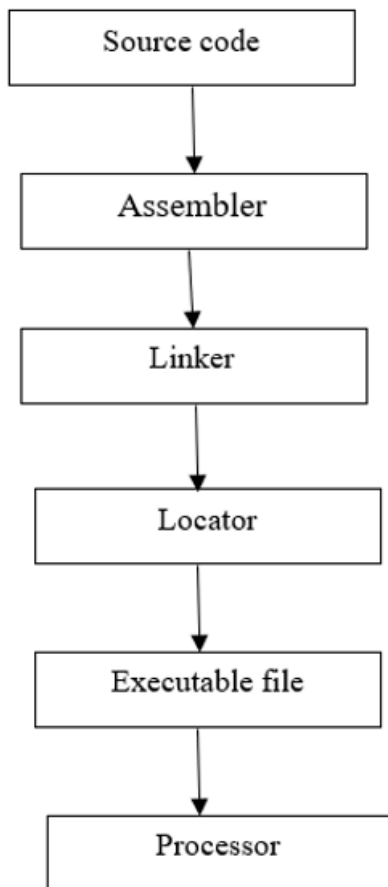


Fig 7: flow of flashing source code to a processor

3.3 Embedded System Software

Embedded system software is created with the purpose of carrying out specific tasks and is usually written in a high-level programming language. This software is then converted into code that can be stored in the hard drive within the hardware. When developing embedded system software, three main factors are considered: availability of system memory, availability of processor speed, and limitations on power dissipation during continuous operation, including stop, run, and wake-up events.

The programmed code must be loaded onto a microprocessor or microcontroller, which serves as the hardware component in charge of carrying out the activities specified in the code, in order to integrate the software with the hardware of an embedded system. Processors only run executable files; source code for embedded devices is typically written in assembly language.

Software and hardware integration is a necessary step in the creation of software for embedded systems. The software can run on the embedded device and perform the desired tasks by transforming the source code into an executable binary image.

3.3.1 Arduino IDE

The Arduino IDE (Integrated Development Environment) is an official program developed by Arduino.cc. It functions as a platform for creating, compiling, and uploading code to the Arduino device.

INTRODUCTION TO ARDUINO IDE

- The open-source Arduino IDE software is primarily used for authoring and compiling code for Arduino modules.
- The official Arduino program offers a simple code generation process, allowing even beginners without prior technical experience to learn the basics.
- Arduino operates on the Java Platform, providing essential functions for debugging, editing, and generating code.
- It supports various Arduino modules like Uno, Mega, Leonardo, and Micro, with microcontrollers on board. Core code, or sketch, generates a Hex File for board control.
- The core code, sometimes referred to as a sketch, written on the IDE platform will eventually produce a Hex File, which is uploaded and sent to the board's controller.
- The IDE environment consists of two main components: the Editor for writing code and the Compiler for compiling and uploading it to the Arduino Module.
- The C and C++ languages are supported in this environment.

Downloading and installing Arduino IDE

- The Arduino website provides software downloads for popular operating systems like Windows, Linux, and MAX, ensuring compatibility with the appropriate version.
- Make sure you have Windows 10 or Windows 11 if you want to download a Windows app. To better serve our development needs, we can also download earlier iterations of the IDE.
- In most cases, we also need to install drivers for the specific development board we are

using. This enables our computers to recognize the modules as development boards and not as a random flash drive.



 **Arduino IDE 2.3.2**

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file

Linux AppImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Fig 8: Arduino IDEs download page

Arduino IDE is divided into 3 parts:

- Menu Bar
- Text Editor
- Output Window/Serial Monitor

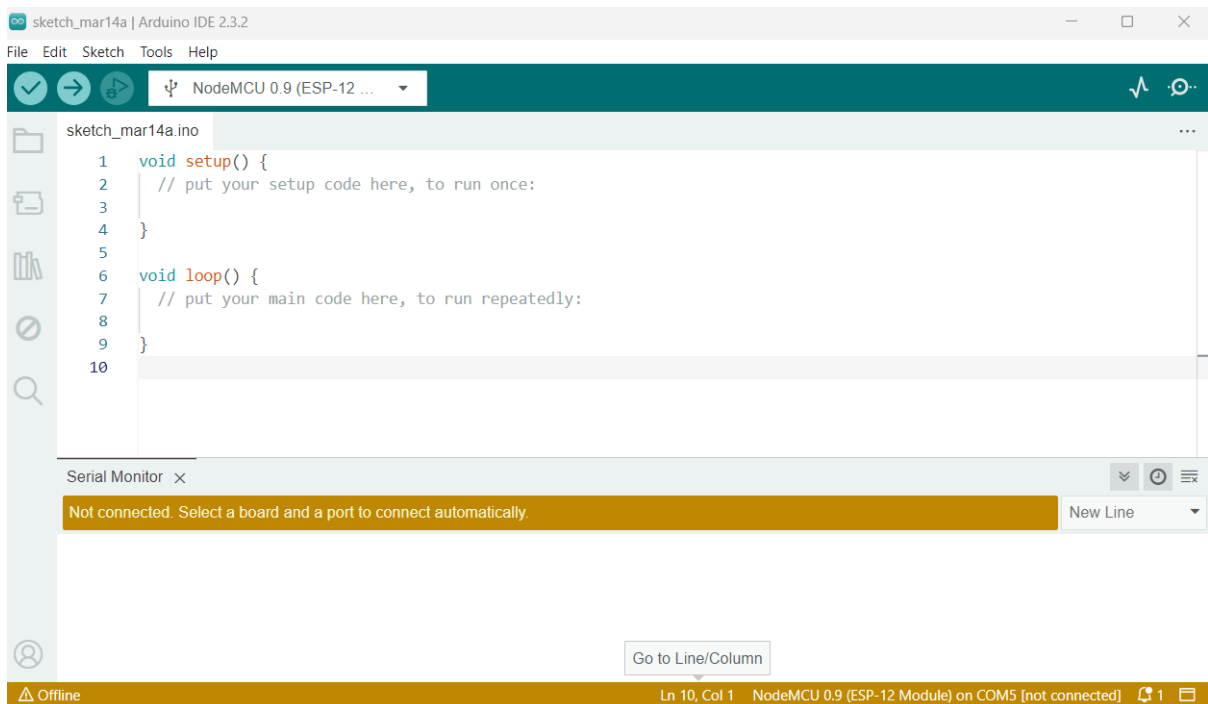


Fig 9: Arduino IDE as seen during startup

The bar appearing on the top is called Menu Bar that comes with five different options as follow

- **File** – The file option allows you to create a new code window or open an existing one, with a table detailing the subdivisions it can be categorized into.
- **Edit** – Used for copying and pasting the code with further modification for font
- **Sketch** – For compiling and programming
- **Tools** – This panel is primarily utilized for testing projects, with the Programmer section being utilized for burning a bootloader to a new microcontroller.
- **Help** – The company offers comprehensive assistance to those who are skeptical about software, from starting the process to troubleshooting.

The 3 Buttons appearing under the Menu tab are connected with the running program as follow.

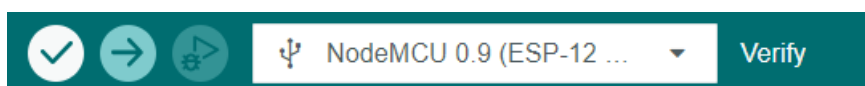


Fig 10: Action Panel

From left to right,

- The first one verifies the written source code for the type of development board we are using.

- The second button, forward arrow, is used to flash the complied source code to the selected development board.
- The drop-down menu is used to select the development board connected to an available COM.

Since every board comes with a different microprocessor, it is necessary to select the correct development board, in order to convert the HLL source code to the specific Assembly code.

Installing Drivers for development board

Drivers for development boards either come directly from the manufacturer or are made by 3rd party enthusiasts. Selection of correct drivers is very crucial since they provide the firmware using which the devices are able to communicate with our systems.

Drivers are executable files whose installation is as simple as installing MS Word. Arduino IDE supports most of the devices natively. Nevertheless, to get the Heltec ESP32 LoRa module working, the manufacturer provides the drivers here: [CP210x Drivers](#)

This LoRa module uses a CP210x chip to connect to a computer through a USB C cable.

Installing ESP32 and NodeMCU development Boards into Arduino IDE

To add development boards such as ESP32 and NodeMCU, we use the menu bar to navigate to: *Files>Preferences*

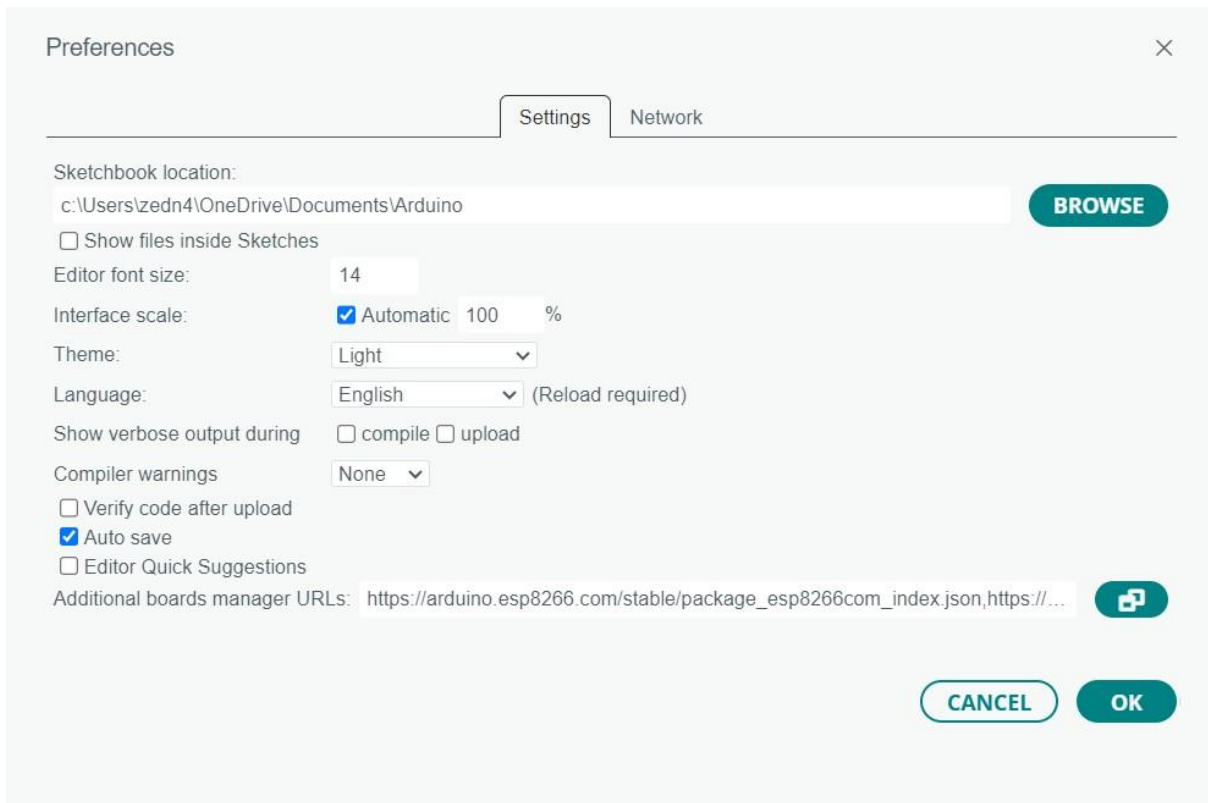


Fig 11: Preferences window

In the additional boards manager URLs column, we paste:
https://arduino.esp8266.com/stable/package_esp8266com_index.json

- Open Boards Manager from Tools > Board > Boards Manager
- Select the version you need from a drop-down box.
- Click *install* button.

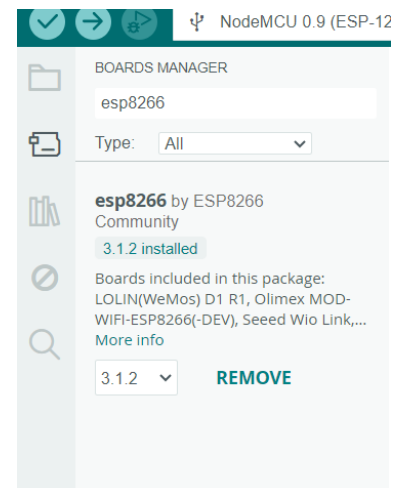


Fig 12: Boards Manager

The Next development board we will need to install is for Heltec ESP32 LoRa Module.

Similarly, we open the preference window, and click the besides the address space to open:

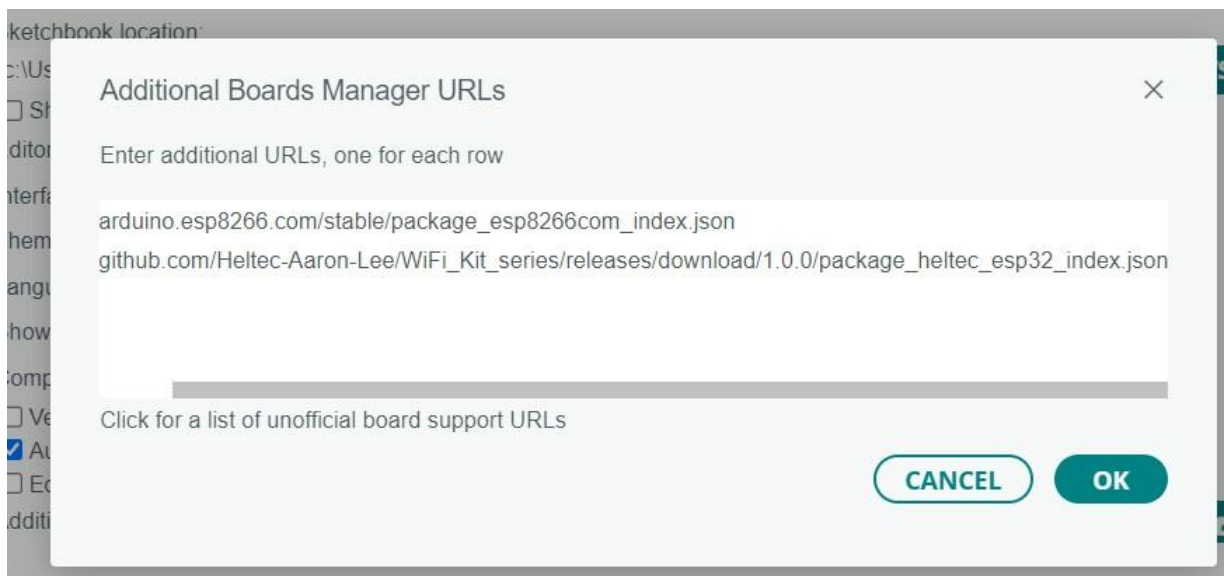


Fig 13: Screen to add additional boards

There we can add the following link in the list:

https://github.com/Heltec-Aaron-Lee/WiFi_Kit_series/releases/download/1.0.0/package_heltec_esp32_index.json

[Lee/WiFi_Kit_series/releases/download/1.0.0/package_heltec_esp32_index.json](https://github.com/Heltec-Aaron-Lee/WiFi_Kit_series/releases/download/1.0.0/package_heltec_esp32_index.json)

Now the board is added, we open the boards manager by navigating to,

Tools>boards>boards manager and install heltec ESP32 series Dev boards.

After installing these two boards, we will be ready to work on our Transmission side modal.

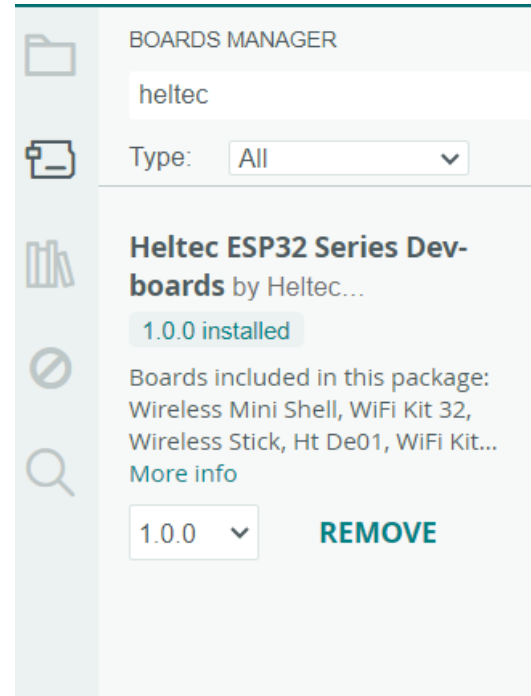
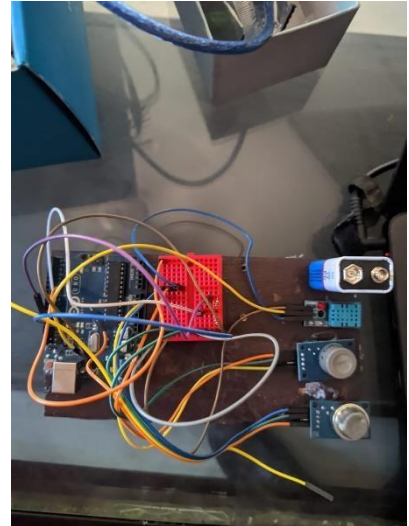


Fig 14: Boards Manager

3.4 Working on Transmission Side Modal

In order to utilize the transmission modal effectively, it is necessary to extract analog data from Carbon Monoxide, and dust sensors. Nevertheless, due to the limitation of having only one analog pin on the Node MCU, we are restricted to reading data from just one analog sensor. To address this issue, we have incorporated an Arduino UNO with the LoRa Module



CH4- Methane Sensor: MQ7

MQ series sensors are resistive sensors that utilize a sensitive layer composed of SnO_2 . When subjected to heat, these sensors have the ability to adsorb different gases onto their surface. The resulting concentration of gases is then measured as a change in voltage, which is subsequently converted into a value representing parts per million.

To make this sensor work as intended, we need to add MQ sensor unified library which facilitates the conversion between voltage and ppm values.

To install the library, following steps were followed:

- Download following library from Git: [MQUnified Library](https://github.com/miguel5612/MQSensorLib.git)
- Navigate to sketch> include library>add .zip library
- Locate the downloaded library and add it.
- Arduino IDE will install it automatically

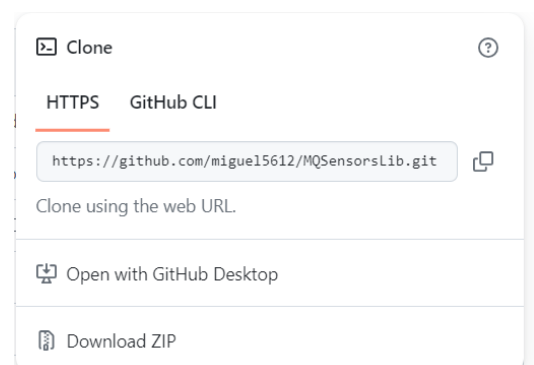


Fig 15: MQ Unified Library

```
#include <MQUnifiedsensor.h>

#define placa "Arduino UNO"
#define Voltage_Resolution 5
#define pin1 A1
#define type1 "MQ-7"
#define ADC_Bit_Resolution 10
#define RatioMQ7CleanAir 27.5
#define PWMPin1 5
```

Fig 16: Defining Properties of MQ sensors

When using MQ sensors, we must set up the necessary parameters. We have to indicate the board we are using, which in this situation is ArduinoUNO. The Voltage Resolution is the operating voltage of our sensor. Its analog pin is linked to pin 5 on Arduino. As the library supports various MQ sensors, we have to specify our MQ sensor, which is MQ7 in this instance. RatioMQ7CleanAir is the ratio that sets the clean air threshold.

After these parameters are defined, we will be able to make an object of MQUnifiedsensor library and use itss function to convert the output voltage into a PPM value. Output of the following experiment is attached below.

[65.00]	v = ADC*5.00/1023.00		0.32		RS = ((5.00*RL)/(Voltage) - RL)		147.38		Ratio = RS/R0		11.72		ratio*a + b		2.36
[65.00]	v = ADC*5.00/1023.00		0.32		RS = ((5.00*RL)/(Voltage) - RL)		147.38		Ratio = RS/R0		11.72		ratio*a + b		2.36
[111.00]	v = ADC*5.00/1023.00		0.54		RS = ((5.00*RL)/(Voltage) - RL)		82.58		Ratio = RS/R0		6.57		ratio*a + b		5.69
[128.00]	v = ADC*5.00/1023.00		0.62		RS = ((5.00*RL)/(Voltage) - RL)		70.55		Ratio = RS/R0		5.61		ratio*a + b		7.23
[136.00]	v = ADC*5.00/1023.00		0.66		RS = ((5.00*RL)/(Voltage) - RL)		65.22		Ratio = RS/R0		5.19		ratio*a + b		8.14
[141.00]	v = ADC*5.00/1023.00		0.69		RS = ((5.00*RL)/(Voltage) - RL)		62.55		Ratio = RS/R0		4.97		ratio*a + b		8.68
[138.00]	v = ADC*5.00/1023.00		0.68		RS = ((5.00*RL)/(Voltage) - RL)		63.60		Ratio = RS/R0		5.06		ratio*a + b		8.46
[113.00]	v = ADC*5.00/1023.00		0.55		RS = ((5.00*RL)/(Voltage) - RL)		80.13		Ratio = RS/R0		6.37		ratio*a + b		5.96
[101.00]	v = ADC*5.00/1023.00		0.49		RS = ((5.00*RL)/(Voltage) - RL)		91.29		Ratio = RS/R0		7.26		ratio*a + b		4.89
[92.00]	v = ADC*5.00/1023.00		0.45		RS = ((5.00*RL)/(Voltage) - RL)		101.80		Ratio = RS/R0		8.09		ratio*a + b		4.14
[86.00]	v = ADC*5.00/1023.00		0.42		RS = ((5.00*RL)/(Voltage) - RL)		108.95		Ratio = RS/R0		8.66		ratio*a + b		3.74
[81.00]	v = ADC*5.00/1023.00		0.39		RS = ((5.00*RL)/(Voltage) - RL)		117.08		Ratio = RS/R0		9.31		ratio*a + b		3.35
[81.00]	v = ADC*5.00/1023.00		0.40		RS = ((5.00*RL)/(Voltage) - RL)		116.30		Ratio = RS/R0		9.25		ratio*a + b		3.38
[77.00]	v = ADC*5.00/1023.00		0.38		RS = ((5.00*RL)/(Voltage) - RL)		122.00		Ratio = RS/R0		9.70		ratio*a + b		3.15
[77.00]	v = ADC*5.00/1023.00		0.37		RS = ((5.00*RL)/(Voltage) - RL)		123.73		Ratio = RS/R0		9.84		ratio*a + b		3.08
[75.00]	v = ADC*5.00/1023.00		0.36		RS = ((5.00*RL)/(Voltage) - RL)		127.32		Ratio = RS/R0		10.12		ratio*a + b		2.95
[74.00]	v = ADC*5.00/1023.00		0.36		RS = ((5.00*RL)/(Voltage) - RL)		129.18		Ratio = RS/R0		10.27		ratio*a + b		2.89
[73.00]	v = ADC*5.00/1023.00		0.36		RS = ((5.00*RL)/(Voltage) - RL)		130.14		Ratio = RS/R0		10.35		ratio*a + b		2.85
[90.00]	v = ADC*5.00/1023.00		0.44		RS = ((5.00*RL)/(Voltage) - RL)		104.30		Ratio = RS/R0		8.29		ratio*a + b		3.99
[129.00]	v = ADC*5.00/1023.00		0.63		RS = ((5.00*RL)/(Voltage) - RL)		69.30		Ratio = RS/R0		5.51		ratio*a + b		7.43
[136.00]	v = ADC*5.00/1023.00		0.66		RS = ((5.00*RL)/(Voltage) - RL)		65.50		Ratio = RS/R0		5.21		ratio*a + b		8.09
[141.00]	v = ADC*5.00/1023.00		0.69		RS = ((5.00*RL)/(Voltage) - RL)		62.81		Ratio = RS/R0		4.99		ratio*a + b		8.62
[144.00]	v = ADC*5.00/1023.00		0.70		RS = ((5.00*RL)/(Voltage) - RL)		61.04		Ratio = RS/R0		4.85		ratio*a + b		9.00

Fig 17: Output for MQ sensors

CO- Carbon Monoxide Sensor: MQ4

Carbon monoxide sensors are used to detect the amount of carbon monoxide in a room. MQ4 sensor is able to detect a lot of gases, thus we need to modify the parameters so it can detect CO.



```
MQ4.setA(1012.7); MQ4.setB(-2.786);  
MQ4.init();
```

MQ4.setA and MQ4.setB is used to prepare the sensor to sense carbon monoxide gas.

The sensor's parameters need to be set similarly as that of MQ7 sensor.

```
/* ===== Hardware Related Macros ===== */  
#define Board ("Arduino UNO")  
#define Pin (A0) //Analog input 4 of your arduino  
/* ===== Software Related Macros ===== */  
#define Type ("MQ-4") //MQ4  
#define Voltage_Resolution (5)  
#define ADC_Bit_Resolution (10) // For arduino UNO/MEGA/NANO  
#define RatioMQ4CleanAir (4.4) //RS / R0 = 60 ppm  
/* ===== Globals ===== */
```

Fig 18: Properties for MQ4 sensor for Unified Sensor

The Output of the following was:

A screenshot of the Arduino IDE Serial Monitor window. The window title is "63 | */". The "Output" tab is selected. The serial data shows the following information:
Contributors: Message (Enter to send message to 'Arduino Uno' on 'COM4')
Sensor: MQ-4
Supply voltage: 5.00 VDC
ADC Resolution: 10 Bits
R0: 10.10 kΩ
RL: 10.00 kΩ
Model: Exponential
MQ-4 -> a: 1012.70 | b: -2.79
Development board: Arduino UNO
Below this, a table of data is displayed, showing ADC values, calculated resistance, and PPM values for various gas concentrations.

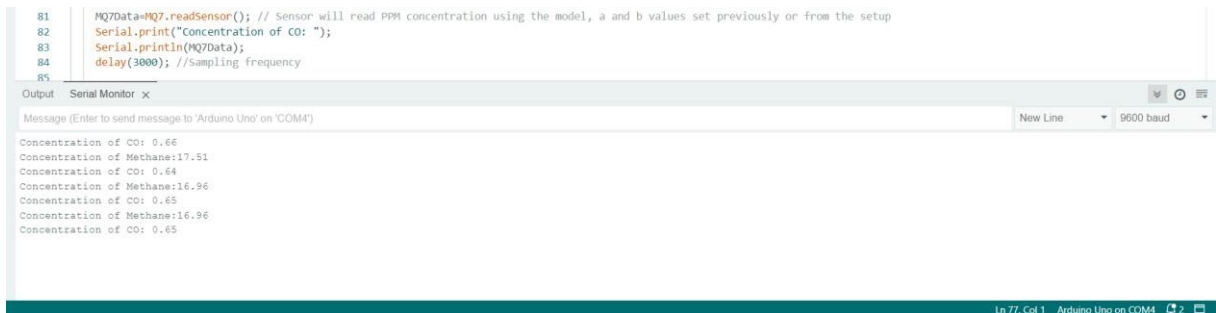
ADC_In	Equation_V_ADC	Voltage_ADC	Equation_RS	Resistance_RS	Eq_Ratio	Ratio (RS/R0)	Equation_PPM	PPM
1114.00	v = ADC*5.00/1023.00	0.55	RS = ((5.00*RL)/(Voltage) - RL)	80.53	Ratio = RS/R0	4.45	ratio*a + b	15.94
1109.00	v = ADC*5.00/1023.00	0.53	RS = ((5.00*RL)/(Voltage) - RL)	84.29	Ratio = RS/R0	4.66	ratio*a + b	13.95
1108.00	v = ADC*5.00/1023.00	0.53	RS = ((5.00*RL)/(Voltage) - RL)	84.29	Ratio = RS/R0	4.66	ratio*a + b	13.95
1105.00	v = ADC*5.00/1023.00	0.52	RS = ((5.00*RL)/(Voltage) - RL)	86.97	Ratio = RS/R0	4.80	ratio*a + b	12.78
1107.00	v = ADC*5.00/1023.00	0.52	RS = ((5.00*RL)/(Voltage) - RL)	86.06	Ratio = RS/R0	4.75	ratio*a + b	13.16
1103.00	v = ADC*5.00/1023.00	0.51	RS = ((5.00*RL)/(Voltage) - RL)	88.84	Ratio = RS/R0	4.91	ratio*a + b	12.05
1104.00	v = ADC*5.00/1023.00	0.51	RS = ((5.00*RL)/(Voltage) - RL)	88.37	Ratio = RS/R0	4.88	ratio*a + b	12.23
1104.00	v = ADC*5.00/1023.00	0.50	RS = ((5.00*RL)/(Voltage) - RL)	89.32	Ratio = RS/R0	4.93	ratio*a + b	11.87
1102.00	v = ADC*5.00/1023.00	0.50	RS = ((5.00*RL)/(Voltage) - RL)	89.80	Ratio = RS/R0	4.96	ratio*a + b	11.69
1102.00	v = ADC*5.00/1023.00	0.50	RS = ((5.00*RL)/(Voltage) - RL)	90.79	Ratio = RS/R0	5.01	ratio*a + b	11.34
1102.00	v = ADC*5.00/1023.00	0.50	RS = ((5.00*RL)/(Voltage) - RL)	90.29	Ratio = RS/R0	4.99	ratio*a + b	11.51
1100.00	v = ADC*5.00/1023.00	0.49	RS = ((5.00*RL)/(Voltage) - RL)	92.30	Ratio = RS/R0	5.10	ratio*a + b	10.83
1098.00	v = ADC*5.00/1023.00	0.48	RS = ((5.00*RL)/(Voltage) - RL)	94.39	Ratio = RS/R0	5.21	ratio*a + b	10.18

Fig 19: Output for MQ4 Sensor

After successfully reading the sensor data through an Arduino, our efforts were directed towards establishing serial communication between Arduino UNO and ESP32.

Both analog sensors were then connected to Arduino UNO at the same time, and we noted the

output. *espSerial.println(AQITest)* was used to send the data to the serial ports of Arduino UNO.



```
81 MQ7Data=MQ7.readSensor(); // Sensor will read PPM concentration using the model, a and b values set previously or from the setup
82 Serial.print("Concentration of CO: ");
83 Serial.println(MQ7Data);
84 delay(3000); //Sampling frequency
85
```

Output Serial Monitor x

Message (Enter to send message to 'Arduino Uno' on 'COM4')

Concentration of CO: 0.66
Concentration of Methane:17.51
Concentration of CO: 0.64
Concentration of Methane:16.96
Concentration of CO: 0.65
Concentration of Methane:16.96
Concentration of CO: 0.65

Ln 77, Col 1 Arduino Uno on COM4

Fig 20: Serial Communication

Our next thing was setting up the NodeMCU to receive data.

Serial.available() and *Serial.read()* are the commands which are used to read the upcoming serial data from Arduino.



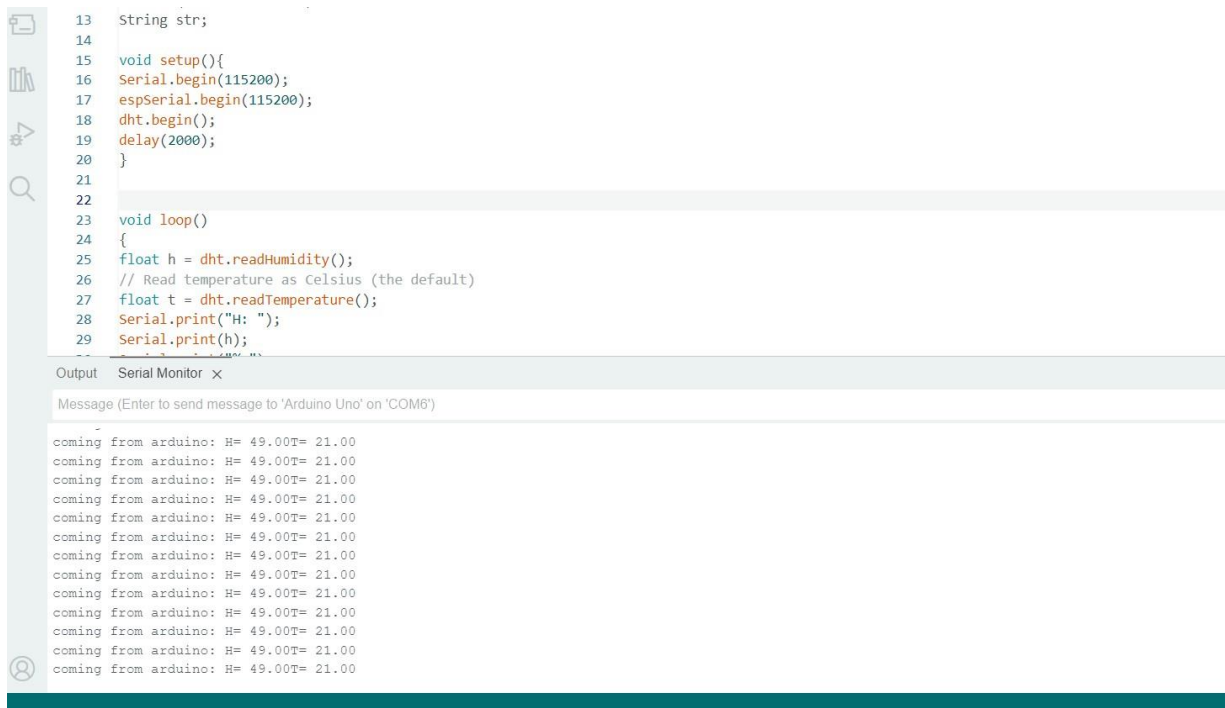
```
Data is coming from Arduino
55.00
Data is coming from Arduino
46.00
Data is coming from Arduino
50.00
Data is coming from Arduino
56.00
Data is coming from Arduino
13.00
Data is coming from Arduino
10.00
Data is coming from Arduino
55.00
Data is coming from Arduino
46.00
```

g pin

Fig 21: Serial Communication Temperature and Humidity

Upon confirming the successful reading of data from the analog sensors on the ESP serial monitor, we proceeded to connect a DHT11 module in order to analyze the temperature and humidity levels of the surrounding environment.

Upon installation of a DHT Library via the library manager feature, we connect the DHT11 to our transmitter configuration. Like the aforementioned sensors, the data is collected on the Arduino board and subsequently transmitted to the ESP using software serial communication. The resulting output of this setup is as follows.



```
13 String str;
14
15 void setup(){
16   Serial.begin(115200);
17   espSerial.begin(115200);
18   dht.begin();
19   delay(2000);
20 }
21
22
23 void loop()
24 {
25   float h = dht.readHumidity();
26   // Read temperature as Celsius (the default)
27   float t = dht.readTemperature();
28   Serial.print("H: ");
29   Serial.print(h);
30 }
```

Output Serial Monitor x

Message (Enter to send message to 'Arduino Uno' on 'COM6')

coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00
coming from arduino: H= 49.00T= 21.00

Fig. 22: Temperature and Humidity data using Serial Communication

The last step in optimizing the transmission mode was to create a cloud server utilizing spreadsheets, and enable data transfer through the WiFi capabilities of NodeMCU to a Google spreadsheet.

In order to transfer data to a Google Sheet, we must first construct the Sheet and then set it up to accept data from the device by tying the Sheet to our device with a Gscript (Google Script). To do this, take the actions listed below:

1. Open docs.google.com in your browser.
2. Click the menu button when the page loads, then choose the Google sheet from the list of applications that drops down.
3. Clicking the above will take you to a new page where you can select to create a new sheet.
4. Give the new spreadsheet an easily recognizable name (document title) when it opens.

After the sheet is created, we make a custom google script to log the data from the webserver into the spreadsheet.

```

1 // Enter Spreadsheet ID here
2 var SS = SpreadsheetApp.openById('1cnQ1A8wAp_drIBNz1ZSH6-YusGUcYRfoDCAgRy2PUhs');
3 var str = "";
4
5
6
7 function doPost(e) {
8
9   var parsedData;
10  var result = {};
11
12  try {
13    parsedData = JSON.parse(e.postData.contents);
14  }
15  catch(f){
16    return ContentService.createTextOutput("Error in parsing request body: " + f.message);
17  }
18
19  if (parsedData != undefined){
20    var flag = parsedData.format;
21    if (flag == undefined){
22      flag = 0;
23    }
24
25    var sheet = SS.getSheetByName(parsedData.sheet_name); // sheet name to publish data to is specified in Arduino code
26    var dataArr = parsedData.values.split(","); // creates an array of the values to publish
27
28    // Default time zone is America/Chicago. Update time zone below with appropriate ID from here: https://developers.google.com/google-ads/api/data/codes-formats#timezone-ids
29    var date_now = Utilities.formatDate(new Date(), "America/Chicago", "yyyy/MM/dd"); // gets the current date
30    var time_now = Utilities.formatDate(new Date(), "America/Chicago", "hh:mm:ss a"); // gets the current time
31
32    var value0 = dataArr[0]; // value0 from Arduino code
33    var value1 = dataArr[1]; // value1 from Arduino code

```

Fig 23: Script for logging data into google sheet

Both the laptop, and the NodeMCU must be connected to the same network inorder for this to work. A Test with random values was first performed and we got the following output.

	A	B	C	D	E	F
1	Date	Time	Value 0	Value 1	Value 2	
2	2023/12/20	11:18:22 PM	8	87	4995	
3	2023/12/20	11:18:12 PM	7	664	22320	
4	2023/12/20	11:18:03 PM	6	767	46443	
5	2023/12/20	11:17:53 PM	5	713	20431	
6	2023/12/20	11:17:44 PM	4	318	85775	
7	2023/12/20	11:17:34 PM	3	243	75702	
8	2023/12/20	11:17:25 PM	2	908	81638	
9	2023/12/20	11:17:15 PM	1	471	77862	
10						
11						
12						
13						
14						
15						

Fig 24: Random values being printed at Spreadsheet

After confirming the legitimacy of the arbitrary data, we proceeded to connect the remaining sensors and initiate the script to retrieve the data from the ESP module.

AQITest and DHT11

File Edit View Insert Format Data Tools Extensions Help

100% 123 Default... 10

E2 51

	A	B	C	D	E	F
1	Date	Time	Temperature	Humidity	AQITest	
2	2023/12/21	1:32:26 AM	19.8	48	51	
3	2023/12/21	1:32:20 AM	19.6	48	57	
4	2023/12/21	1:32:13 AM	19.4	48	46	
5	2023/12/21	1:32:07 AM	19.4	48	10	
6	2023/12/21	1:32:01 AM	19.4	48	53	
7	2023/12/21	1:31:55 AM	19.4	48	46	
8	2023/12/21	1:31:48 AM	19.4	48	46	
9	2023/12/21	1:31:42 AM	19.4	49	10	
10	2023/12/21	1:31:36 AM	19.4	49	51	
11	2023/12/21	1:31:30 AM	19.4	49	46	

Fig 25: Temperature, Humidity and AQI being printed in spreadsheet server

3.5 Testing DSM501a Dust sensor- PM2.5 and PM10

We started working on DSM501a Dust sensor. It is an interesting sensor with a heating filament inside it. The heat generated by the filament, heats up the air around it, establishing an air flow between the cold air of the surrounding. Once the air is inside, Light/Laser gets scattered by the dust particles in air. This scattered pulse is then returned to the sensor. The time taken for the pulse to return corresponds to an estimated number of PM2.5 and PM10 particles in air.

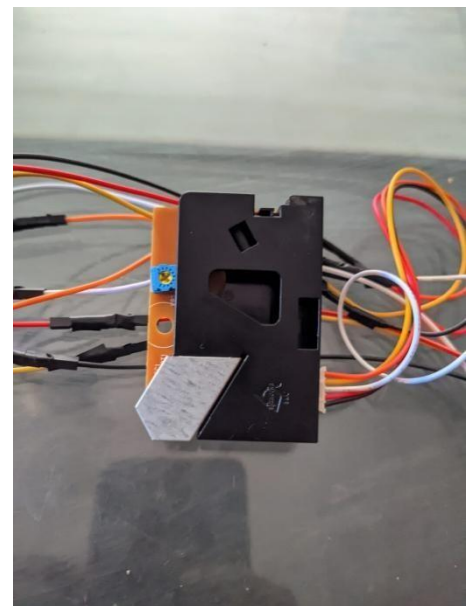


Fig 26: DSM501a Dust sensor

Once the following connections were made, test code was flashed into the Arduino UNO. The results obtained were:

- Arduino Vin-> DSM501a +5V
- Arduino A0 -> DSM501a PM 10 pin
- Arduino A1 -> DSM501a PM 2.5 pin
- Arduino GND -> DSM501a GND pin

```

53      concentration = 0.001915 * pow(ratio,2) + 0.09522 * ratio - 0.04884; //Cal
54      */
55      float conPM1 = calculateConcentration(lowpulseoccupancyPM1_30);

```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 0.9 (ESP-12 Module)' on 'COM5')

lowpulseoccupancy:0	ratio:0.00	Concentration: 0.00
PM1 -0.05 PM25 -0.05		
lowpulseoccupancy:0	ratio:0.00	Concentration:-0.05
lowpulseoccupancy:0	ratio:0.00	Concentration:-0.05
PM1 -0.05 PM25 -0.05		
lowpulseoccupancy:0	ratio:0.00	Concentration:-0.05
lowpulseoccupancy:0	ratio:0.00	Concentration:-0.05
PM1 -0.05 PM25 -0.05		
lowpulseoccupancy:0	ratio:0.00	Concentration:-0.05
lowpulseoccupancy:0	ratio:0.00	Concentration:-0.05
PM1 -0.05 PM25 -0.05		

Fig 27: Non converted values of Dust sensors

The concentration values we received were then converted into mg/m^3 using the two ratios provided in the documentation. The mathematical formula used in this conversion was:

```

float ratio = (lowpulseInMicroSeconds/1000000.0)/30.0*100.0;
float concentration = 1.1*pow(ratio,3)-3.8*pow(ratio,2)+520*ratio+0.62;

```

Our next step was to Integrate the LoRa Modules into our system. After researching for a while, we ended up using Ra-02 modules by semtech. They are popular modules based on SX1278 which have been tested by the masses and have well defined libraries which let us integrate the system seamlessly with our project.

3.6 LoRa Modules

The first step to Integrating LoRa with the rest of the project is to Install the necessary libraries. We will be using LoRa by Sandeep which supports a wide array of LoRa chips by different

manufacturers.

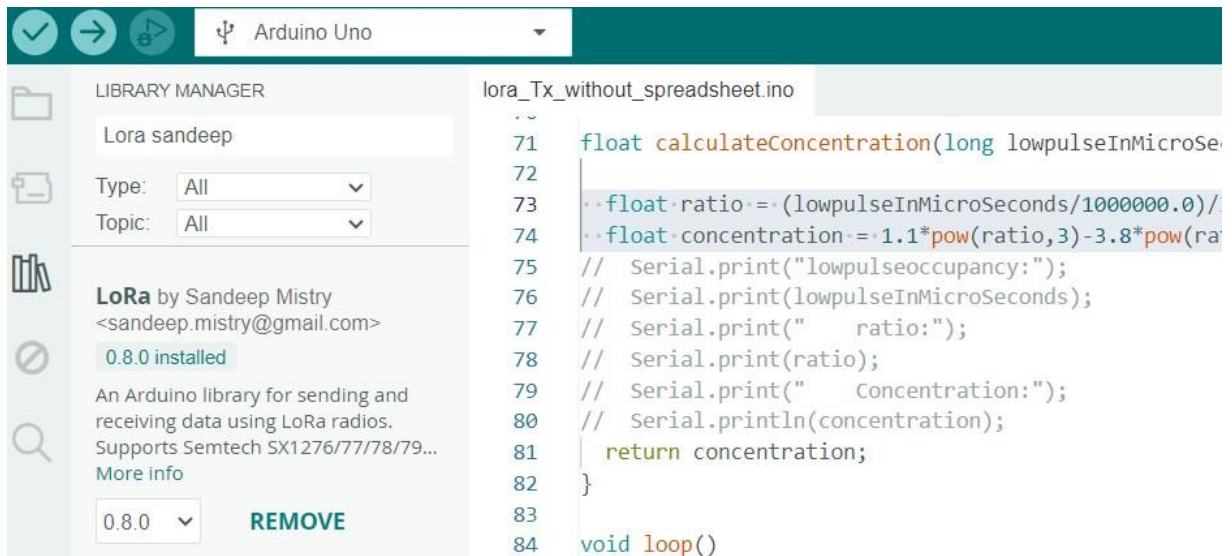


Fig 28: LoRa library by Sandeep Mistry

Next step comes the interfacing of Ra-02 LoRa module using Sandeep's Library. The Following are the connections we made to integrate the LoRa Module with the Tx Side

SX1278 LoRa Chip	Arduino UNO Connections
VCC	3.3V
GND	GND
SCK	SCK
MISO	MISO
MOSI	MOSI
NSS	10
NRST	9
DIO0	2

For the LoRa Module to initialize, we need to make sure to set Pins for Lora. Which can be done through the following command.

NSS, NRESET, and DIO0 pins can be changed by using `LoRa.setPins(ss, reset, dio0)`

Another thing to keep in mind to make the LoRa modules communicate is that we need to program the two modules with same settings. The settings we need to make sure are same as follows:

- 1 **Frequency:** Since LoRa Modules don't have a specific address for communication, it is absolutely necessary to make sure that the two modules are working on the same sending and listening frequency.
- 2 **Bandwidth:** It can be between 137 MHz to 525 Mhz. For our project, we are using a bandwidth of 125.
- 3 **Coding Rate:** It is the ratio of error corrected bits to Information carrying bits. They are used to prevent us from errors during the package transmission. This value is set to 4/5
- 4 **Preamble Length:** Before the actual data payload, the preamble consists of a sequence of known symbols. It fulfills various roles such as synchronization, estimating frequency offset, and initializing automatic gain control.
- 5 **Sync Word:** LoRa doesn't have an address to differentiate and communicate with each other. So on the same frequency, we need to set the two LoRa Modules with the same sync word for the modules to communicate. These sync words are specific to different chips. In our program we are using 0xA5

6 Spreading Factor

7 IQ Inversion

Since we are using identical LoRa Modules, most of our settings will be same, but we still need to set the frequency and Sync word for effective communication.

Before we connected the entire project with our LoRa Modules, we needed to see if the modules were sending and receiving data. In LoRa library, we can also print two parameters, RSSI and SNR.

- RSSI is the ratio which comments on the signal strength of the communication between

the two devices.

- SNR is the signal to Noise Ratio.

Both these parameters can be used to debug the LoRa communication and to test the effective length of the communication

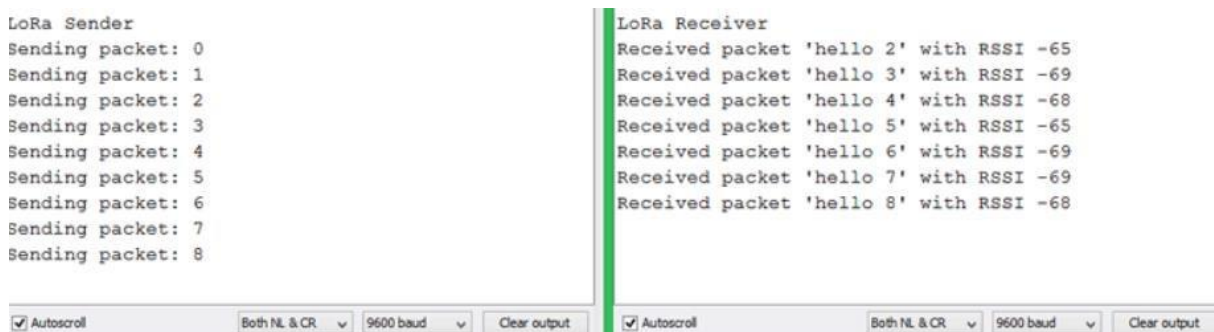


Fig 29: Sending data using LoRa

The Code used at the Tx Side:

```
#include <SPI.h>

#include <LoRa.h>

int counter = 0;

void setup() {

  Serial.begin(9600);

  while (!Serial);

  Serial.println("LoRa Sender");

  if (!LoRa.begin(433E6)) {

    Serial.println("Starting LoRa failed!");

    while (1);

  }

  LoRa.setTxPower(20);
}

void loop() {
```

```

Serial.print("Sending packet: ");

Serial.println(counter);

// send packet

LoRa.beginPacket();

LoRa.print("hello ");

LoRa.print(counter);

LoRa.endPacket();

counter++;

}

```

The Code used at the Rx Side:

```

#include <SPI.h>

#include <LoRa.h>

void setup() {

  Serial.begin(9600);

  while (!Serial);

  Serial.println("LoRa Receiver");

  if (!LoRa.begin(433E6)) {

    Serial.println("Starting LoRa failed!");

    while (1);

  }

}

void loop() {
// try to parse packet
  int packetSize = LoRa.parsePacket();

```

```

if (packetSize) {

    // received a packet

    Serial.print("Received packet ");

    // read packet

    while (LoRa.available()) {

        Serial.print((char)LoRa.read());

    }

    // print RSSI of packet

    Serial.print(" with RSSI ");

    Serial.println(LoRa.packetRssi());

}

}

```

The only thing left for us to test before integrating everything is the 0.96 OLED. It will be interfaced with the ESP32 at the receiving end to show the LoRa Rx Data.

The libraries used in this case are: Adafruit_SSD_1306 and Adafruit_GFX.

The Adafruit_SSD_1306 library requires: SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET

We tested the Oled display with the sample code provided in the library, and then tried printing “hello world” on the screen as the quintessential code.

After printing everything on the display, it is essential to call the display.display(); function, to clear the print buffer.

Code for printing “Hello World” on OLED

```

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 64 // OLED display height, in pixels
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

```

```

void setup() {
    Serial.begin(115200);

    if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // Address 0x3D for 128x64
        Serial.println(F("SSD1306 allocation failed"));
        for(;;);
    }
    delay(2000);
    display.clearDisplay();

    display.setTextSize(1);
    display.setTextColor(WHITE);
    display.setCursor(0, 10);
    // Display static text
    display.println("Hello, world!");
    display.display();
}

void loop() {

}

```

The output observed on the screen was:



Fig 30: OLED Testing

3.7 Arduino JSON

Now, that each of our component is tested separately, what we need to do now is integrate every component together. One thing to consider however is the data type of our transmitted data. We can transmit struct using LoRa but it is much convenient to do so in Strings. To make the parsing of strings easier, we use a Java library for Arduino IDE. This library is called: Arduino_JSON

We can find the library here: <https://github.com/bblanchon/ArduinoJson>

Arduino JSON is a library that can easily parse and create JSON (JavaScript Object Notation) data for Arduino and other microcontroller platforms.

JSON is a different data exchange format used mainly for exchanging data between servers and web applications, but is also useful for exchanging data between systems and other objects used.

The JSON library provides functions for parsing JSON strings into data structures that can be easily manipulated in Arduino sketches, as well as functions for creating JSON strings from data structures. This allows Arduino projects to easily communicate with web servers and other devices and even parse configuration data stored in JSON format.

3.8 Final Project Assembly

Connections made for the transmitter:

Sensor Pins	Arduino Pins
DSM500a PM2.5	A1
DSM500a PM10	A0
DHT	A3
NSS	D10
MOSI	D11
MISO	D12
SCK	D13
GND	GND
3.3v	3.3v
RST	D9
DIO0	D6

Code for Transmission Module:

```
#include <Wire.h>
#include <SFE_BMP180.h>
```

```

#include <SPI.h>
#include <DHT.h>
float t,h;
#define DHTPIN A3

#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

#include <LoRa.h>
#define ss 10
#define rst 9
#define dio0 2

SFE_BMP180 pressure;
double T,P,p0,a;
double baseline; // baseline pressure
#define ALTITUDE 1655.0

#include<string.h>
#define PM1PIN A0//DSM501A input D6 on ESP8266
#define PM25PIN A1
byte buff[2];
unsigned long durationPM1;
unsigned long durationPM25;
unsigned long starttime;
unsigned long endtime;
unsigned long sampletime_ms = 1000;
unsigned long lowpulseoccupancyPM1 = 0;
unsigned long lowpulseoccupancyPM25 = 0;
int i=0;
float conPM1=0;
float conPM25=0;
////////////////////
void setup()
{

    Serial.begin(9600);
    pinMode(PM1PIN,INPUT);
    pinMode(PM25PIN,INPUT);
    starttime = millis();
    dht.begin();
    while (!Serial);
    Serial.println("LoRa Sender");
    LoRa.setPins(ss, rst, dio0);
    if (!LoRa.begin(433E6))
    {
        Serial.println("Starting LoRa failed!");
        delay(100);
    }
}

```

```

    while (1);
}
delay(1000);
Serial.println("AT");
if (pressure.begin())
Serial.println("BMP180 init success");
else
{
    Serial.println("BMP180 init fail (disconnected?)\n\n");
    while(1); // Pause forever.
}
// delay(1000);

}

float      calculateConcentration(long      lowpulseInMicroSeconds,      long
durationinSeconds){

    float ratio = (lowpulseInMicroSeconds/1000000.0)/30.0*100.0; //Calculate the
ratio
    float concentration = 1.1*pow(ratio,3)-3.8*pow(ratio,2)+520*ratio+0.62;
// Serial.print("lowpulseoccupancy:");
// Serial.print(lowpulseInMicroSeconds);
// Serial.print("    ratio:");
// Serial.print(ratio);
// Serial.print("    Concentration:");
// Serial.println(concentration);
    return concentration;
}

void loop()
{
    h = dht.readHumidity();
    t = dht.readTemperature();
    if (isnan(h) || isnan(t))
    {
        Serial.println("Fail to read from DHT sensor!");
        return;
    }

    durationPM1 = pulseIn(PM1PIN, LOW);
    durationPM25 = pulseIn(PM25PIN, LOW);

    lowpulseoccupancyPM1 = lowpulseoccupancyPM1 + durationPM1;
    lowpulseoccupancyPM25 = lowpulseoccupancyPM25 + durationPM25;

    endtime = millis();

```

```

    if ((endtime-starttime) > sampletime_ms) //Only after 30s has passed we
    calculalte the ratio
    {
        conPM1 = calculateConcentration(lowpulseoccupancyPM1,30);
        conPM25 = calculateConcentration(lowpulseoccupancyPM25,30);
        Serial.print("PM1 ");
        Serial.print(conPM1);
        Serial.print(" PM25 ");
        Serial.println(conPM25);
        lowpulseoccupancyPM1 = 0;
        lowpulseoccupancyPM25 = 0;
        starttime = millis();
    }

```

```

baseline = getPressure();

```

```

String data = "";
data+= "{";
data+= "\"anloga\":";
data+= "\"" +String(t)+"\"",";
data+= "\"anlogb\":";
data+= "\"" +String(h)+"\"",";
data+= "\"anlogc\":";
data+= "\"" +String(baseline)+"\"",";
data+= "\"anlogd\":";
data+= "\"" +String(conPM1)+"\"",";
data+= "\"anloge\":";
data+= "\"" +String(conPM25)+"\"";
data+= "}";
Serial.print('\r');
Serial.print(data);
delay(10);
Serial.print('\n');
LoRa.beginPacket();
LoRa.print(data);
LoRa.endPacket();
}

```

```

double getPressure()
{
    char status;
    double T,P,p0,a;

```

```

    status = pressure.startTemperature();
    if (status != 0)

```



```

{
  // Wait for the measurement to complete:

  delay(status);

  status = pressure.getTemperature(T);
  if (status != 0)
  {

    status = pressure.startPressure(3);
    if (status != 0)
    {

      delay(status);

      status = pressure.getPressure(P,T);
      if (status != 0)
      {
        return(P);
      }
      else{}
    }
    else{}
  }
  else{}
}
else{}
}

```

OUTPUT

```

09:15:54.070 -> { "anloga": "32.80", "anlogb": "49.00", "anlogc": "989.24", "anlog
09:15:55.016 -> { "anloga": "32.80", "anlogb": "49.00", "anlogc": "989.22", "anlog
09:15:55.937 -> PM1 278.69 PM25 0.62
09:15:56.003 -> { "anloga": "32.80", "anlogb": "49.00", "anlogc": "989.20", "anlog
09:15:56.941 -> { "anloga": "32.80", "anlogb": "48.00", "anlogc": "989.24", "anlog
09:15:57.805 -> PM1 125.47 PM25 0.62
09:15:57.875 -> { "anloga": "32.80", "anlogb": "48.00", "anlogc": "989.24", "anlog
09:15:58.813 -> { "anloga": "32.80", "anlogb": "48.00", "anlogc": "989.20", "anlog
09:15:59.750 -> PM1 195.42 PM25 0.62
09:15:59.824 -> { "anloga": "32.80", "anlogb": "48.00", "anlogc": "989.24", "anlog
09:16:00.762 -> { "anloga": "32.80", "anlogb": "48.00", "anlogc": "989.21", "anlog
09:16:01.660 -> PM1 145.58 PM25 0.62
09:16:01.693 -> { "anloga": "32.80", "anlogb": "48.00", "anlogc": "989.24", "anlog
09:16:02.642 -> { "anloga": "32.80", "anlogb": "47.00", "anlogc": "989.25", "anlog
09:16:03.529 -> PM1 67.92 PM25 0.62
09:16:03.564 -> { "anloga": "32.80", "anlogb": "47.00", "anlogc": "989.24", "anlog
09:16:04.571 -> { "anloga": "32.80", "anlogb": "47.00", "anlogc": "989.24", "anlog

```

Fig 31: Serial Monitor Output at Sender Side

Here,

analoga = temperature

analogb = humidity

analogc = pressure

PM1 = PM10

PM25 = PM2.5

Connections made at the Receiving Module

Sensor Pins	ESP32 Pins
GND	GND
NSS	D5
MOSI	D23
MISO	D19
SCK	D18
3.3v	3.3 V

RST	D14
DIO0	D2
SDA	D22
SCK	D21

Code for Receiving Module:

```
#include <LoRa.h>
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

#define ss 5
#define rst 14
#define dio0 2
String LoRaData;
#define LED 4
String inString = "";
String data_read = "";
int val = 0;

#include <Arduino_JSON.h>
String inputData = "";
boolean data_complete = false;
String vala;
String valb;
String valc;
String vald;
String vale;
void setup()
{
  Serial.begin(9600);
  inputData.reserve(200);
  pinMode(LED,OUTPUT);
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C))
  {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;);
  }
}
```

```

delay(2000);
display.clearDisplay();

display.setTextSize(1);
display.setTextColor(WHITE);
display.setCursor(0, 10);

display.println("LoRa Receiver");
display.display();

while (!Serial);
Serial.println("LoRa Receiver");

LoRa.setPins(ss, rst, dio0);

while (!LoRa.begin(433E6))
{
    Serial.println(".");
    delay(500);
}
LoRa.setSyncWord(0xA5);
Serial.println("LoRa Initializing OK!");
}

void loop()
{
    int packetSize = LoRa.parsePacket();
    if (packetSize)
    {
        Serial.println("Received packet");
        while (LoRa.available())
        {
            digitalWrite(LED, HIGH);
            int inChar = LoRa.read();
            inString = inString + (char)inChar;
        }
        demoParse();
        //    lcd.clear();
        //data_read=inString;
        inString = "";
        LoRa.packetRssi();

        display.clearDisplay();
        display.setTextSize(1);
        display.setTextColor(WHITE);
        display.setCursor(20, 0);
        display.println("LoRa Receiver");
    }
}

```

```

        display.setTextSize(1);
        display.setTextColor(WHITE);
        display.setCursor(0, 5);
        display.println(LoRaData);
        display.print("TM: ");
        display.println(vala);
        display.display();

        display.setTextSize(1);
        display.setTextColor(WHITE);
        display.setCursor(0, 15);
        display.println(LoRaData);
        display.print("Hm: ");
        display.println(valb);
        display.display();

        display.setTextSize(1);
        display.setTextColor(WHITE);
        display.setCursor(0, 25);
        display.println(LoRaData);
        display.print("Am Pr: ");
        display.println(valc);
        display.display();

        display.setTextSize(1);
        display.setTextColor(WHITE);
        display.setCursor(0, 35);
        display.println(LoRaData);
        display.print("Pm 1.0: ");
        display.println(vald);
        display.display();

        display.setTextSize(1);
        display.setTextColor(WHITE);
        display.setCursor(0, 45);
        display.println(LoRaData);
        display.print("Pm 2.5: ");
        display.println(vale);
        display.display();
    }
    Serial.println(data_read);
    digitalWrite(LED, LOW);
}

void demoParse()
{
    Serial.println("parse");
    Serial.println(vala);

```

```

Serial.println("====");
JSONVar myObject = JSON.parse(inString);
if (JSON.typeof(myObject) == "undefined")
{
    Serial.println("Parsing input failed!");
    return;
}
Serial.print("JSON.typeof(myObject) = ");
Serial.println(JSON.typeof(myObject));

if (myObject.hasOwnProperty("anloga"))
{
    Serial.print("myObject[\"anloga\"] = ");
    vala = (const char*) myObject["anloga"];
    Serial.println(vala);
    Serial.println((const char*) myObject["anloga"]);
}
if (myObject.hasOwnProperty("anlogb"))
{
    Serial.print("myObject[\"anlogb\"] = ");
    valb = (const char*) myObject["anlogb"];
    Serial.println(valb);
    Serial.println((const char*) myObject["anlogb"]);
}
if (myObject.hasOwnProperty("anlogc"))
{
    Serial.print("myObject[\"anlogc\"] = ");
    valc = (const char*) myObject["anlogc"];
    Serial.println(valc);
    Serial.println((const char*) myObject["anlogc"]);
}
if (myObject.hasOwnProperty("anlogd"))
{
    Serial.print("myObject[\"anlogd\"] = ");
    vald = (const char*) myObject["anlogd"];
    Serial.println(vald);
    Serial.println((const char*) myObject["anlogd"]);
}
if (myObject.hasOwnProperty("anloge"))
{
    Serial.print("myObject[\"anloge\"] = ");
    vale = (const char*) myObject["anloge"];
    Serial.println(vale);
    Serial.println((const char*) myObject["anloge"]);
}
Serial.print("myObject = ");
Serial.println(myObject);
Serial.println();

```

}

OUTPUT

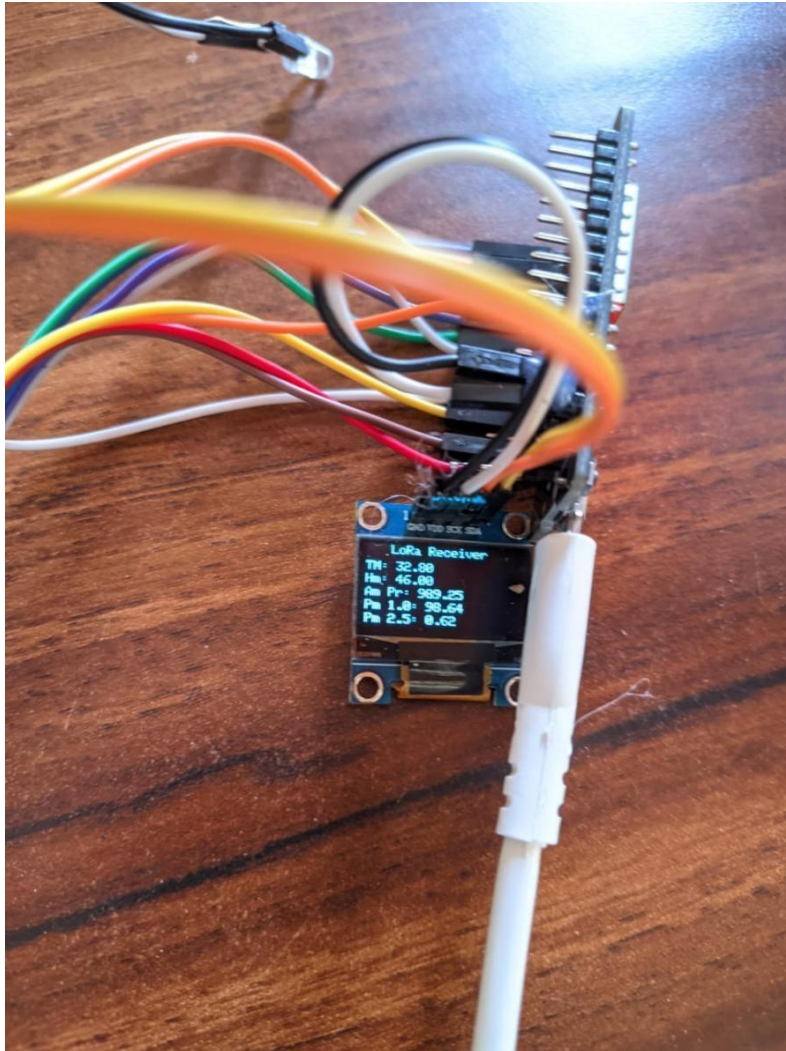


Fig 32: Output on the Oled on the receiving side

CHAPTER 4

RESULT AND WORK DONE

The primary goal of our weather system project was to explore the potential of LoRa as a communication medium under diverse environmental conditions. Our aims encompassed assessing its endurance, Signal to Noise Ratio (SNR), and signal strength across varying mediums and weather scenarios. We aimed to track Humidity, Temperature, Pressure, and PM particles for research purposes, delving into how wind impacts these variables. Moreover, we delved into the ramifications of atmospheric pressure shifts on wind dynamics. A crucial aspect involved juxtaposing our temperature data with meteorological department records and gauging sensor precision against each other.

Throughout this endeavor, we garnered valuable insights into serial, UART, and LoRa communication protocols, alongside mastering data collection, visualization, and storage methods using spreadsheets. We meticulously managed costs without compromising functionality, ensuring alignment with our research objectives. Ultimately, our findings advance comprehension of LoRa's role in weather monitoring systems, emphasizing the criticality of precise sensor data in meteorological investigations.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

CONCLUSION

In summary, real-time environmental monitoring has shown to be feasible with the use of a LoRa-integrated advanced air quality monitoring system. We were able to gather and send data on a variety of atmosphere contaminants, such as particulate matter, volatile organic compounds, and carbon monoxide, among others, by deploying sensor nodes outfitted with LoRa transmission devices. According to our research, LoRa technology has a number of benefits when measuring air quality, including capacity, low power consumption, and long-range ability to communicate. This makes it possible to set up an affordable but efficient monitoring network, especially in places that are difficult to access or remote where traditional surveillance techniques might not be feasible.

Better guided choices for pollution control and public health management have been made possible by the data gathered from the monitoring system, which has also offered insightful information on patterns and trends in air quality. We can improve our capacity to track air quality in real-time and help create more sustainable and healthy settings by utilizing LoRa technology.

In the future, further development and research efforts should be devoted to improve the precision and dependability of the sensors and broadening the monitoring network's capabilities. The system's usefulness for individuals who make decisions can also be increased by interaction with data analytics and visualization tools. In summary, the LoRa-based air quality monitoring system is an important achievement into attaining our objectives of better living conditions and healthier breathing. It also shows tremendous potential for tackling the difficulties of surveillance of the environment in today's world.

FUTURE SCOPE

Improved Sensor Technology: As sensor technology continues to progress, air quality assessments will become more precise, sensitive, and specific. To enable more thorough pollutant detection, research and development activities should concentrate on multi-sensor array integration, calibration, and downsizing.

Integration with Internet of Things Platforms: By combining the LoRa-based monitoring system with cloud services and IoT platforms, data processing, storage, and analysis capabilities can be improved. We are able to derive more meaningful conclusions from the gathered data and create early warning systems for incidents related to air quality by utilizing machine learning algorithms and predictive analytics

REFERENCES

- [1] Harsh Gupta; Dhananjay Bhardwaj; Himanshu Agrawal; Vinay Anand Tikkiwal; Arun Kumar. “An IoT Based Air Pollution Monitoring System for Smart Cities” 2019 IEEE International Conference on Sustainable Energy Technologies and Systems (ICSETS).
- [2] Vaishnavi Lakhara; Priya Kurade; Tejaswini Pawar; Arti Chougule; Mrs. Medha Asurlekar. “Real Time Weather Monitoring System Implementation Based on Internet of Things” Proceedings of the 4th International Conference on Advances in Science & Technology (ICAST2021).
- [3] Evariste Twahirwa* Kambombo Mtonga* Desire Ngabo Santhi Kumaran “A LoRa enabled IoT-based Air Quality Monitoring System for Smart City” ; IEEE world AI IOT Congress 2021.
- [4] Ravi Sai Manasa¹ , Patlolla Aarthi Reddy² , Tudi Nandini Reddy³ , Gaddam Ramakrishna Goud⁴ , Koleti Bhulakshmi⁵ “LoRa based Wireless Weather Station with WebServer” ; April 2023 | IJIRT | Volume 9 Issue 11 | ISSN: 2349-6002.
- [5] G. Jayasree, G. Anjali, D. Lakshmi Saraswathi, Syed. Shamshujeya “LoRa BASED SMART CITY AIR QUALITY MONITORING SYSTEM” ; 2023 IJCRT | Volume 11, Issue 4 April 2023.
- [6] S M Abiduzamman* Hashma Mansor* Teddy surya Gunawan* Robiah Ahmad* “Real time outdoor air quality monitoring system”; 2021 IEEE 7th international conference on smart instrumentation, measurement and applications.
- [7] Virendra Barot; Viral Kapadia “Air Quality Monitoring System using IoT” ; 2020 ComPE.
- [8] Indranil Sarkar; BijoyPal; Arnab Datta; Sandip Roy “WiFi Based Portable weather station for monitoring temperature, relative humidity, pressure, precipitation, wind speed and direction” ; 2019 ICT4SD.
- [9] Toju Babalola, Abayomi Babalola, Mayowa olokun “Development of an ESP-32 Microcontroller based weather reporting system” ;July 2022 Journal of engineering research and reports.
- [10] Eko Murdyantoro, Imron Rosyadi, Ridlo Setiawan, Azis WW Nugraha “Prototype weather station uses LoRa wireless connectivity infrastructure”; journal of physics conference series 2019.
- [11] Bharat S. Chaudhari ^{1,*} Marco Zennaro ² and Suresh Borkar ³ “LPWAN Technologies: Emerging Application Characteristics, Requirements, and Design

Considerations”; Future Internet 6 March 2020.

[12] Hamouda Mohammed .H.M1, Sameep Dave,”Wireless weather monitoring system using Arduino DUE and GSM Technology”, IRJETvol.5, ISO 9001-2008, pp.746,747, 2018.

[13] Aris Munandar*, Hanif Fakhurroja, Muhammad Ilham Rizqyawan, Rian Putra Pratama, “Design of Weather Monitoring System Based on Mobile Application sing Automatic Weather Station”, IEEE transaction pp.44,46,2017.

[14] V.J. Hodge ,S.O’Keefe and A.Moulds, “ Wireless Sensor Network for Condition Monitoring in the Railway Industry: A survey,” IEEE Transactions on intelligent transportation systems Pp.1088, 2015.

[15] Mary Nsabagwa; Isaac Mugume; Robert Kasumba; Joshua Muhumu, “Condition Monitoring and Reporting Framework For Wireless Network-based Automatic Weather Stations”, Endorsed transaction on internet of things 4 pp.1,8, 2018.